

TyrSecure

An eBPF-based anti-cheat solution for Linux

9th Semester Project





Title:

TyrSecure - An eBPF-based anti-cheat solution for Linux

Theme:

Pre-Specialisation in Computer Science

Project Period:

Fall Semester 2025

Project Group:

CS-IT9 and SW9, Group 2

Participant(s):

Eske Klintø Larsen
Mathias Holm Mariager
Thormod Jakob Schøn Merstrand
Faustas Anulis
Andreas Hove Paludan

Supervisor:

Brian Nielsen

Page Numbers:

49

Date of Completion:

January 7, 2026

Abstract:

As kernel-level anti-cheat solutions have become increasingly controversial due to privacy and stability concerns, this project explores the efficiency of leveraging Linux eBPF as an alternative. The research begins by mapping the landscape of video game exploits and defenses through an attack/defense tree. Afterwards, it details the current kernel-level anti-cheat solutions and its potential advantages and downsides. Then the intricacies of eBPF are presented, leading to the question of whether eBPF can effectively replace kernel modules. This will be explored through a set of requirements that were deemed to be necessary to have eBPF as an effective replacement. These requirements serve as the benchmark for TyrSecure, an eBPF based anti-cheat software. This is followed by an evaluation of TyrSecure. Afterwards, there will be a discussion of various elements of the project. From the project as a whole, it was found that while the current implementation of TyrSecure shows promising results, more work would be needed to conclude whether eBPF is an effective replacement for kernel module. Future work should focus on restructuring the architecture of the current implementation, along with integrating secure boot and testing in a real environment.

Contents

1	Introduction	1
2	Problem Analysis	2
2.1	Definition of Cheating	2
2.2	Classification of Cheats	3
2.2.1	Infrastructure Level Cheats	4
2.3	Attack/Defence Tree	4
2.4	Operating Systems Used for Gaming	6
2.4.1	Windows	6
2.4.2	Linux	7
2.4.3	Android	9
2.4.4	Choice of OS	10
2.5	The Structure of Linux	11
2.6	User-Level and Kernel-Level Anti-Cheat Systems	12
2.6.1	The Advantages of Kernel-Level Anti-Cheat Systems	12
2.6.2	The Downsides of Kernel-Level Anti-Cheat Systems	13
2.7	eBPF	14
2.7.1	The Possibilities and Limitations of eBPF	16
2.7.2	CO-RE	16
3	Problem Statement	17
4	Methodology	18
5	Requirements	19

5.1	Loadable Kernel Modules	19
5.2	File Manipulation	20
5.3	File Integrity	21
5.4	Memory Scanning	21
5.4.1	User-Level Memory Access	22
5.4.2	Kernel-Level Memory Access	23
5.5	Virtual Machine	23
5.6	Secure Boot	24
5.7	Summary of Requirements	27
6	Implementation	28
6.1	The Memory Agent	29
6.2	The Memory Handler	30
6.3	The Memory Tracker	32
6.4	Kernel Module Tracker	35
6.5	Project Structure	36
7	Evaluation	37
7.1	Test Setup	37
7.2	Test Method	39
7.3	Test Result	39
8	Discussion	42
8.1	Limitations of the Test	42
8.2	Unsolved Requirements	42
8.3	Limitations of eBPF	43
9	Conclusion	44
10	Future Work	45
10.1	Architecture Redesign and Feature Expansion	45
10.2	Virtual Machine Detection	46
10.3	Integration With Secure Boot	47

10.4 Integration With Wine/Proton	47
10.5 Priorities for Future Development	49

Chapter 1

Introduction

The video game industry has become a major force in the entertainment industry, with the video game industry being worth more than triple the movie and music industry combined [47]. In this industry some of the most popular types of games are the competitive online games, with the most popular of these hitting monthly player counts of 20-40 million players [71]. The competitive aspects of these games draw in players to honor their skills and try to reach the top of players. But some people want to take the shortcut to the top. Competitive video games has seen a rise in players using dishonest methods to gain an advantage, the so called cheaters. One of these is the game Rainbow Six Siege X that in just one month after their June 2025 release saw an increase in cheating of almost 30% [11]. This is just one of the many video games suffering from cheaters, and while many countermeasures exist, new cheating methods are constantly being developed. Motivated by our own experiences with anti-cheat and the growing cheating problem, it lead us to wanting to aid in the development of anti-cheat. This project will first cover our research into the field of cheating in video games, along with the current ways to handle the problem. After this we will lay the groundwork for our proposed solution to cheating in the field and the intricacies of it.

Chapter 2

Problem Analysis

2.1 Definition of Cheating

Video games have been around for a long time and cheating in video games is still a major problem to this day [24]. Video games involve user interaction with UI and input devices to generate visual feedback. A video game will have set boundaries for what is the intended way for a player to interact with the game and its elements.

However, some players want to break free from these boundaries, enter what is known as cheating. A common way to describe cheating could be that: *Cheating is an unfair advantage that goes beyond normal standards and is not intended to be part of the gameplay.* However, the exact definition of cheating differs between game development companies as criteria varies based on what each company defines as cheating behavior. Such differentiation could be caused by many reasons, some examples include: 1) Different game genres have different forms of cheating 2) New cheats are being developed as old cheats are being prevented.

Esports games are becoming a major factor that shape new narratives of how cheating is defined [19]. Esports is commonly defined as competitive gaming in an amateur or a pro league that is facilitated by electronic systems and can involve ladders / tournaments and cash prizes [26]. As esports gained popularity, cheating in multiplayer games has become a strongly recognized issue as it affects the game's ecosystem and all players involved. The motive for cheating in multiplayer games is usually to gain an unfair advantage over other players or to match the skill/impress others to gain respect within the community. In single player games, cheating is more accessible and many developers provide cheat codes [19] to speed up in-game progression [9], so if they choose to cheat, they are only cheating their own machine and it primarily affects the individual player's experience [19].

Cheaters can be socially framed into different types. A "doper/fraudster" is defined as a player who competes in esports tournaments and uses cheats to enhance their performance to win competitions that could involve prizes. Another definition of a "doper/fraudster" is a streamer using restricted software that has commitment to their viewers and tries to maintain audience's respect by showing off their performance. A "frustrated" player typically uses hacks as an emotion coping strategy or due to low self-esteem and a "slacker" cheats due to lack of skill and unwillingness to invest time into the game to build real skills [9].

Cheating can have many consequences for other players, for example, players unfairly losing in-game ranking, a negative impact on the in-game economics, or it can create an unfair competition, to name a few. These consequences can be a major problem for game development companies, as cheating in their games can create a bad experience for legitimate players[65]. This can ultimately have financial consequences for the company both in income loss in form of players quitting the game but could also create a negative perception of the company as a whole.

2.2 Classification of Cheats

According to The and Khan's cheat classification [73], cheating in online games can be categorized into game level cheats, application level cheats, protocol level cheats, and infrastructure level cheats.

Game Level Cheats

This category includes any insecurity in the game logic, implementation, policy, or rules. This kind of cheating can be exploiting bugs, collusion, and time cheating (altering the in-game clock). These methods can give players an advantage, but since they are, intentionally or not, in the game and available to anyone, it is not really as severe as the following categories. They can be prevented by thoroughly testing the game before release, and most of the time developers can also patch these once they know about them.

However, these relatively simple cheats can occasionally have a large impact on an online game. An example is the 2013 Diablo III duplication glitch, where players discovered a method to duplicate gold, which can be traded for other players' items and even real money. The exploit led to an imbalance in the in-game economy, forcing the developer to pause trading while they fixed the bug. Still, it left many honest players unhappy, as their gold had been heavily devalued even after the issue was fixed [52].

Application Level Cheats

This category is defined as modifying the data files, or interfering with the memory of the running game. This includes cheats, such as using Cheat Engine [12] to modify memory, editing configuration files, DLL/process injection, running modified binaries, and bots. To prevent these cheats, developers should move critical game logic and state to an authoritative server. Client side, the game should detect cheating by checking for injected modules, altered memory regions, and debuggers.

Protocol Level Cheats

This kind of cheating is defined as interfering with packets sent and received by the game, which may be duplicated, modified, inserted, or destroyed by an attacker. Replay attacks and packet manipulation fall under this category. The main prevention methods are server validation of network requests and using TLS.

2.2.1 Infrastructure Level Cheats

This category is about modifying hardware or driver software to gain an advantage. Powerful and difficult-to-detect cheats such as hardware direct memory access (DMA) attacks, that enables aimbots and wallhacks, are in this category. Preventing these forms of cheating requires clever game design and strong anti-cheat systems on both client and server.

2.3 Attack/Defence Tree

The different types of cheating we had encountered throughout our research were collected and used to produce an Attack/Defence tree. This was done to systematically document types of cheating we had encountered along with potential countermeasures for them. The structure of the tree was based on the works of Kordy et. al [33] that feature refinements and countermeasures. Refinements are the child nodes of some parent nodes, and can essentially in this case be seen as sub-goals of what the parent node is trying to achieve. Here Kordy et. al [33] further divide refinements into disjunctive and conjunctive ones. Disjunctively refined parent nodes can achieve their goal if at least one of their child nodes achieve its goal, while conjunctively refined parent nodes need all their children to achieve their goal for the parent node to achieve its own goal. Countermeasures is the other type of connection used in the tree, and are present between attack nodes (red circles) and defense nodes (green squares). Using this structure the following Attack/Defence tree was created:

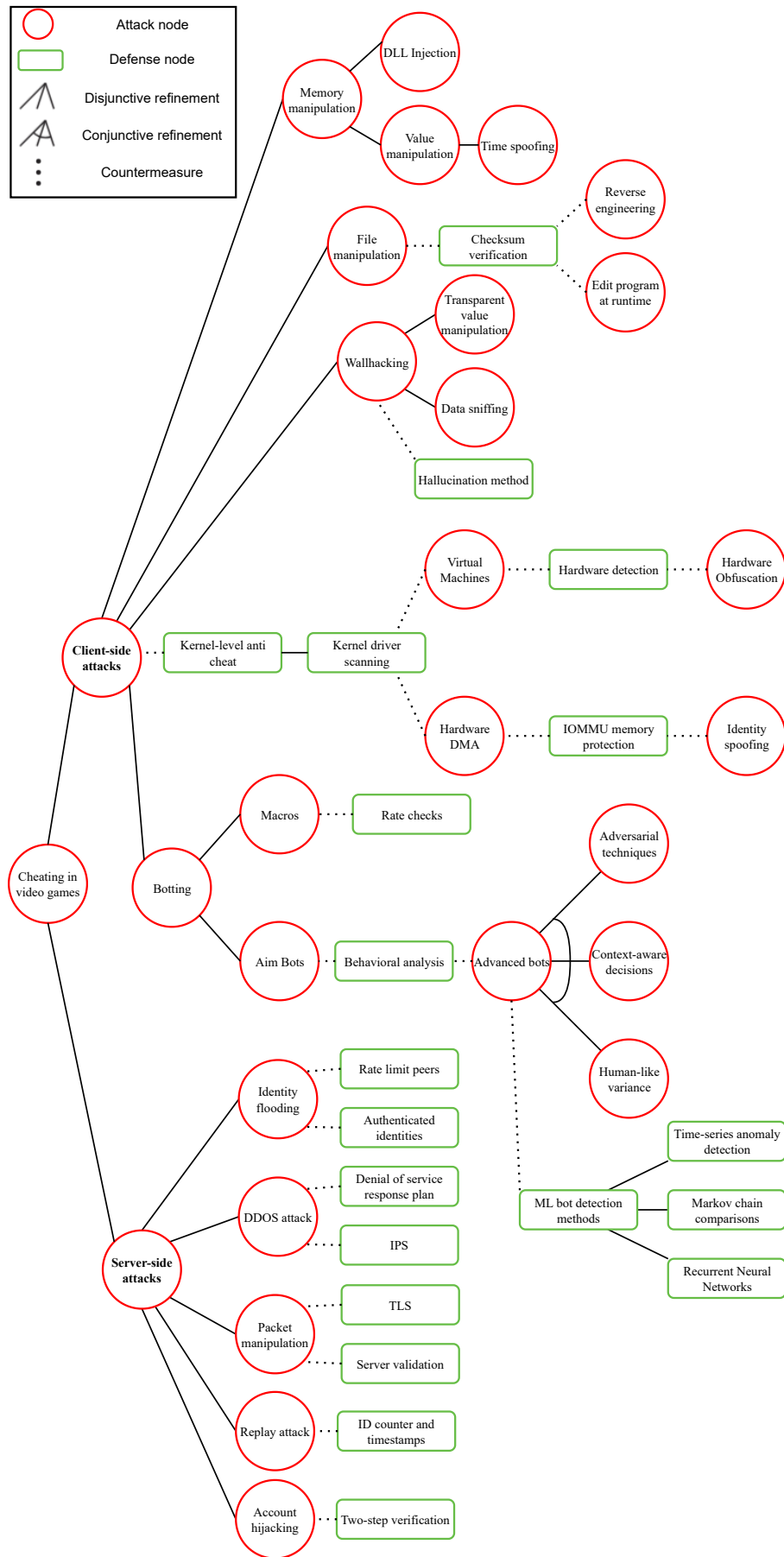


Figure 2.1: Attack/Defence tree of cheating in video games. The two main branches studied are marked in bold

The nodes connected to the root node that make up the tree is the two attack node branches of Client-side attacks and Server-side attacks. We expanded these two nodes with their refinements and countermeasures until we had an Attack/Defence tree that we found summed up our current knowledge of the research field.

From the Attack/Defence tree we found that some areas of particular interest for us were that of the two defense nodes "Behavioral analysis" and "Kernel-level anti cheat" as well as the attack node "Hardware DMA". "Behavioral analysis" was interesting since it would be a way to combat aim bot cheating, a type of cheating plaguing shooter games and currently pretty hard to detect [42]. "Hardware DMA" was interesting since it is a type of cheating that currently is almost impossible to detect and on the rise, with webpages like "dma-cheats.com" [17] even selling DMA cards to private consumers. We did however, ultimately end up going with researching kernel-level anti cheat (KLAC) solutions further, since this would also include looking at hardware DMAs anyway. The field of kernel-level anti cheat is also a field of interest for the gaming industry as a whole, with big game companies like Riot Games [60] and their kernel-level anti cheat solution "Riot Vanguard" [39] being just one example of the variety of current kernel-level anti cheat solutions on the market. Furthermore, there are still some challenges to be resolved within this field, like how transparent the anti cheat should be and most kernel-level anti cheats being guarded by game companies, with only a few open source solutions being available.

2.4 Operating Systems Used for Gaming

When it came to what operating system (OS) to pursue making anti cheat for we had some different options. While it differs what games work for specific OS's we chose to look into Windows, Linux and Android in the following sections.

2.4.1 Windows

According to the largest distributor of PC games, Steam, 95.4% of their users play on Windows[78]. Its popularity means that most games and existing anti-cheat solutions are built for Windows. Some of the most well-known commercial anti-cheat systems for Windows include Riot Vanguard, Easy Anti-Cheat (EAC) and BattlEye. In this section, we will take a closer look at EAC and Riot Vanguard, as well as explore a non-commercial open-source alternative.

EAC is a service offered by Epic Games as part of Epic Online Services. It includes cheat prevention on server and client by blocking common cheating techniques such as reading and writing the game process memory. Epic Games may also create detections for common cheating tools that are used across multiple games, but because maintaining these detections requires ongoing work from Epic, this is not guaranteed [22]. Any developer can integrate it into their game, making it a good choice for smaller studios since building an anti-cheat system from scratch is time-consuming and expensive. It is free to use, but it is also possible to buy additional features such as support, game-specific cheat detection, and machine learning models that can detect aim-assists in shooter games.

Riot Vanguard is Riot Games own custom anti cheat software. The program consists of a user-level component and a kernel level driver. The user mode component will actively monitor the system while a Riot game is running, handling all of the anti cheat detections. The kernel mode driver is then used to validate memory, system state and that the users device has not been tapered with in any way. The driver is run at start up of the computer to ensure that any low level cheats is not loaded beforehand [59]. This means that the kernel-mode driver will run passively on the computer upon start up, which could for some users feel unsafe. However, Riot games claims the driver is not used to send any information back to them, and has made it possible for users to disable the driver if wanted, but the computer would then have to restart to enable the driver again to ensure it can be trusted. Although there has been some push-back on the intrusiveness of Vanguard, Riot Games have found the system to be an effective way to reduce cheating in their games [58].

As mentioned, there also exist a few open source anti-cheat systems that incorporate kernel-mode drivers. One example is UltimateAntiCheat [2], which, similar to EAC is a generic anti-cheat system that is not specific to only one game. An interesting decision is that the kernel driver is not included in the public repository. The author explains that he does not see any point in making it public because 1) most GitHub users will not be able to sign it, 2) open source kernel-mode drivers are easier to exploit, and 3) he does not want to assume any liability if something goes wrong.

Several big gaming titles such as Valorant, League of Legends, EA Sports FC, have made it a requirement for user devices to have Secure Boot and Trusted Platform Module (TPM) 2.0 enabled to play some or all parts of their game. TPM is a specialized hardware component and can be used to store cryptography keys that can help ensure the OS and firmware is authentic and have not been tampered with. The TPM module is also used for different Windows features like Windows Hello and BitLocker and is an requirement for Windows 11. The enforced incorporation of such technologies in Windows gives anti-cheat developers an extra tool to ensure trust on the client [49].

On Windows it has been proven by companies that the implementation of KLAC systems is an effective way to prevent cheating. Windows' large market share mean that companies are willing to invest in expensive anti cheat solutions and also that there are a lot of people investing time and resources in creating cheats for different games, many of those available to the public. This gives us a test environment for a possible solution, where we can test our implemented cheating countermeasures, without having to create our own test environment from scratch. This could be an advantage of choosing Windows, since it would minimize the work load in creating a testing environment for the anti-cheat software. It is important to note that such test does not ensure a general security measure since that would require some kind of formal proof.

2.4.2 Linux

Linux, first released in 1991, is a UNIX-based operating system originally derived from the UNIX [69] system, which was developed in 1969. Since its release, Linux has continued to evolve, and recent developments such as Windows-to-Linux translation layers called Wine, have significantly increased software compatibility[64]. One notable advancement comes from Valve, a corporation known for developing video games and the Steam game distribution platform, which has recently focused on supporting Linux gaming. Valve expanded upon the existing Wine translation software with their own project, Proton, to

increase compatibility for gaming on Linux by improving controller support and performance [81]. As a result, Linux can now run a large majority of games [21]. However, the growing prevalence of anti-cheat systems in modern games has introduced new challenges, limiting Linux gaming in certain areas.

Distribution Fragmentation

Linux exists in a wide range of distributions (distros), each with its own kernel configuration, libraries, and system architecture [37]. This fragmentation barrier complicates the development and maintenance of a unified anti-cheat solution that functions reliably across all variants [35]. However, Valve has developed its own Linux-based distribution targeted for gaming, SteamOS. It is based on Arch Linux, a lightweight and customizable Linux distribution [31]. This consolidation around a gaming-focused Linux platform offers a potential path toward standardization, providing a centralized target for anti-cheat development and reducing the fragmentation barrier that has historically discouraged developers from adopting Linux.

Growing Linux Adoption for Gaming

The introduction of SteamOS and broader gaming support for Linux have made it a more attractive option for gamers. As a result, many companies are starting to manufacture SteamOS driven handheld devices that provide users with a Linux ecosystem.

Another technology on the rise is cloud gaming, and providing anti-cheat could make Linux a viable cloud gaming hosting option. Moreover, dissatisfaction among end users with proprietary operating systems, particularly, perceived bloat in Windows has pushed many users to seek alternatives [51]. Additionally, many users are moving toward Linux for its transparency and control, valuing an open-source operating system where they can trust that their data is not being misused [66]. Another aspect of using Linux is that it is free to use and does not require an activation license, making it an ideal solution for individuals and businesses to minimize their software expenses [74].

Legacy Hardware Support

As hardware ages, many manufacturers discontinue support for older devices. In contrast, Linux often maintains support for legacy hardware, allowing users to continue using their hardware [36]. It is beneficial for users who are unable to afford or unwilling to purchase new hardware. As a result, users may switch to Linux, extending the lifespan of their devices. In regions with limited economic resources, where access to new hardware is restricted, Linux presents a viable and accessible alternative, highlighting its potential for market growth in these demographics.

Lack of Anti-Cheat Solutions

Linux treats the kernel in an open-source way, where users can modify the kernel without any restrictions. However, that increases the difficulty of anti-cheat development [75]. Anti-cheat development on Linux is sparse. As of today, there are still very few Linux kernel-level anti-cheat solutions available. We were unable to find any mature open-source projects, and only a small number of closed-source options exist, such as Easy Anti-Cheat (EAC) and BattlEye [75].

Many Linux users are particularly cautious about software that interferes with the kernel or modifies hardware behavior [18]. Some users may request access to source code, enabling them to verify the software's safety and ensure their privacy is not compromised. While some users are more relaxed about these issues, the Linux community places a high value on control, openness, and trustworthiness, which can create additional challenges for intrusive software such as proprietary anti-cheat systems.

Summary

In summary, Linux has evolved from a UNIX-based system into a versatile, open-source platform that continues to gain traction in gaming. Recent developments such as Wine, Proton, and SteamOS have significantly improved software and gaming compatibility, positioning Linux as an alternative option to proprietary operating systems. However, the fragmented nature of Linux distributions and the challenges of implementing reliable anti-cheat solutions across diverse system architectures remain substantial obstacles.

Despite these limitations, Linux offers advantages for its open-source nature, extensive hardware support, and growing industry adoption through devices powered by SteamOS. These factors contribute to Linux's increasing demand for gaming and emerging areas such as cloud gaming. Continued collaboration between developers, hardware manufacturers, and the open-source community is essential to overcome existing barriers, promote standardization in order to provide an alternative platform for gaming.

2.4.3 Android

When it comes to the mobile gaming space, Android would be the most relevant OS, with Android devices making up 68.7% of the mobile gaming market share [46]. Mobile gamers also make up 54% of the gaming population [41], meaning we would be able to target a majority of gamers with an Android solution. Mobile gaming is also flourishing when it comes to the competitive gaming aspects, with "Esports Charts" reporting the two mobile games "Honor of Kings" and "PUBG Mobile" to have had the third and fourth biggest esports prize pools respectively in 2024 [23]. The competitive gameplay is inviting for cheaters, and in 2022 the creators behind the anti-tamper system Denuvo found that of the top 50 mobile games (70% of these being online multiplayer games), 86% of the titles had a cheat available for the game [30]. They also found that only 22% of the developers behind the examined titles had implemented anti-cheat solutions [30]. This would mean that anti-cheat solutions are still something that has to be explored further in mobile gaming.

Currently one of the ways mobile gaming developers can combat cheating is through the use of Tencent Cloud's mobile gaming anti-cheat solution called Anti-Cheat Expert (ACE). ACE is the anti-cheat solution implemented in the previously mentioned game "Honor of Kings" and aims to protect the game by, for example, verifying the integrity of game memory [72]. Furthermore, it can detect the presence of a virtual machine and also make the player quit the game if it detects tampering with the game speed [72].

Another anti-cheat practice being deployed in the mobile gaming space is root detection. Rooting is the practice of obtaining admin-privileges on a phone, typically done by using a root management application like SuperSU or Magisk. Having a rooted phone gives the user access to bypass Android's built-in security restrictions, this way also obtaining kernel-level access. Rooting opens the way for cheating and therefore popular mobile games like Pokémon GO have implemented root detection, in this particular case, to prevent GPS spoofing. There is however, ways to get around root detection. Magisk for example have a function called "Magisk Hide" that let users chose apps to hide the root from [44]. Furthermore, there exists tools like Apktool that can decompile applications, making it so users can identify the code behind the root detection and change it before recompiling the application [29].

A not so explored part of anti-cheat for mobile games is kernel-level anti-cheat. There is however, reasons for the lack of kernel-level anti-cheats. Android devices by default ship with locked bootloaders, which prevent any modification of the kernel or loading of unsigned kernel code [5]. This would mean that we, just like the cheaters, would have to root the phone to actually implement kernel-level anti-cheat. While rooting can technically allow a user to install custom kernel modules or even replace the kernel entirely, this is also the reason why kernel-level anti-cheat is not a practical solution for commercial Android games. Even with Android's introduction of GKI (Generic Kernel Image), which provides a more stable kernel core across devices [4], developers still cannot distribute arbitrary kernel modules because all modules must be signed with OEM or Google keys. Additionally, many device-specific drivers remain outside the generic kernel core, meaning an anti-cheat module that needs low-level hardware access would still fail on some devices. Combined with locked bootloaders, Verified Boot, and Androids app sandboxing, these restrictions makes it so that kernel-level anti-cheat cannot be deployed safely or universally.

2.4.4 Choice of OS

We chose not to work further with Android, this was due to various reasons one being that the design of Android is less inviting for kernel manipulation, and the distribution of it requires users to also be technical by rooting their devices. Another reason is that creating a kernel-level anti-cheat solution for Android would be hard as mentioned in section 2.4.3, which would mean that a user-level solution would be the most doable. This has however, already been explored by a variety of developers on the Android platform.

This led us to focus on the PC OS's used for gaming. Throughout this research we have found that while Windows is the operating system where a majority of PC gamers reside, Linux as a gaming OS is on the rise. Linux gaming is however still lacking on the anti-cheat side of things, due to many developers choosing to focus mainly on Windows compatibility for their anti-cheat solutions. Therefore, to further aid the development of Linux as a gaming OS, we chose to pursue making a kernel-level anti-cheat that would be compatible with specifically Arch Linux due to it's rise from being the distro SteamOS is built on.

2.5 The Structure of Linux

The Linux operating system is organized into a layered architecture designed for stability and security [70]. Fundamentally it consists of a division between what is called user space and kernel space. This separation ensures that individual programs cannot interfere with the core operating system or the physical hardware directly, preventing a single buggy application from crashing the entire machine.

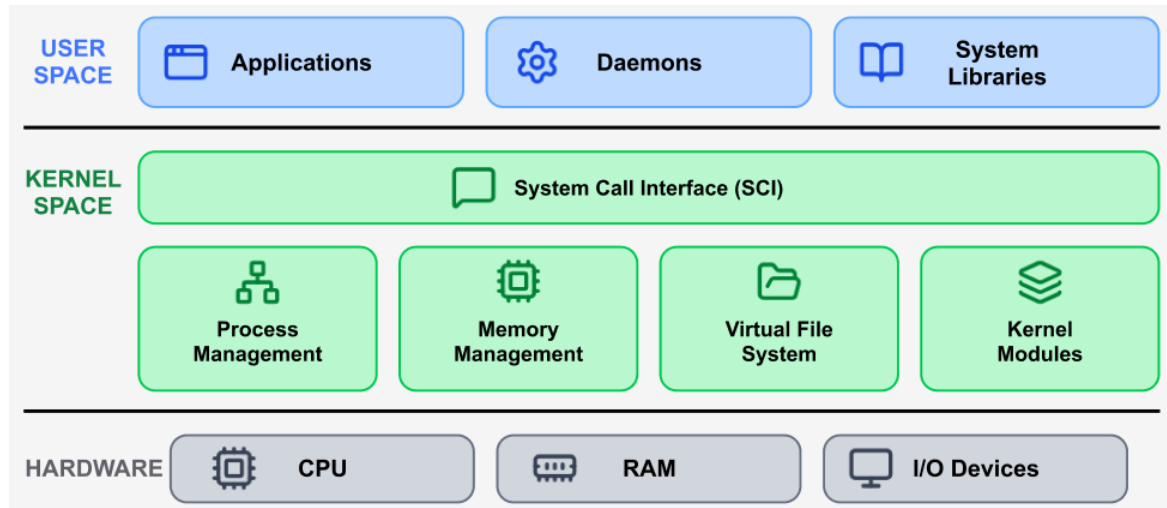


Figure 2.2: The layered Linux structure divided into: user space, kernel space and hardware.

User Space

On the top layer of the architecture we have the user space. This is the environment where all user level applications operate, ranging from web browsers and text editors to background services known as Daemons [70]. Daemons are specialized processes that run silently in the background to handle persistent tasks like network management (networkd), printing (cupsd), or system logging. Because user space is restricted, processes running here are isolated from one another. To perform tasks that require hardware access, such as saving a file to a disk or sending data over the internet, these processes rely on system libraries (a common one being glibc). These libraries provide a standard set of tools that translate high-level programming instructions into a format the kernel can understand.

Kernel Space

The next layer of the architecture is the kernel space. The first part of this space is the System Call Interface (SCI). SCI acts as an gateway between user space and kernel space, when a user space process needs to perform a task, such as opening a file or sending a network packet [70]. This is done trough "system calls" that triggers the CPU to switch from a restricted "user mode" to a privileged "kernel mode," allowing the kernel to take over and safely execute the requested task on the application's behalf. Since the Linux kernel is a monolithic core, it means that it contains all the primary logic for managing the system's resources in a single, high-privileged area. This is things like process management that schedules which tasks get CPU time, memory management that handles RAM

allocation, and the Virtual File System that manages data storage. Another key feature of the kernel layer is its utilization of kernel modules. These modules can be plugged into or unplugged from the kernel at runtime and for example allows the kernel to support new hardware, like a new mouse or a Wi-Fi card, without needing to be entirely rewritten or restarted.

The Hardware Layer

Finally we have the hardware layer. This layer consists of the actual physical electronic components like the CPU, RAM and I/O devices such as hard drives. This layer has no actual intelligence and simply executes the binary instructions and electrical signals that it receives [70]. This is where the kernel comes into play, translating the complex needs of User Space applications into the precise physical actions required by the hardware.

2.6 User-Level and Kernel-Level Anti-Cheat Systems

Anti-cheat systems can be implemented at the previously mentioned two privilege levels: user-level and kernel-level. User-Level Anti-Cheat (ULAC) systems refer to implementations solely on the user-level, and therefore runs with the same privilege as other user applications, including video games. This means ULAC systems relies on system calls to the kernel to query relevant information such as processing states or access to the memory. This limits ULAC systems visibility of the system/device to what the kernel exposes to it, which could be a potential security risk, since it would be unable to detect cheating methods running at a higher privilege than the ULAC system itself. Kernel-Level Anti-Cheat (KLAC) systems can mitigate this security risk, by operating with kernel-level privileges. Implementing a kernel module as part of the anti-cheat system also comes with some risk in regards to crashes, driver conflicts and overall performance issues. Therefore, KLAC systems is often a combination of a user-level and kernel-level implementation, where a kernel module would be able to monitor the system, and enhance the trust of the system calls made by the user-level implementation of the KLAC system.

2.6.1 The Advantages of Kernel-Level Anti-Cheat Systems

In kernel mode the software will have full privilege over the users device, while in user mode the software will have the same privilege as other applications, such as games, on the device. This current structure means that it is hard for user-level anti-cheat software to prevent cheat developers from accessing or manipulating critical game data, if their cheat software runs on kernel-level [54]. To counter cheating threats that use kernel-level privilege to bypass the natural limitations of user-level anti-cheat software, game developers could implement their anti-cheat software at kernel-level. Implementing kernel-level anti-cheat (KLAC) systems, makes it significant harder for cheaters to do unauthorized actions interfering with the games, even when doing more advanced cheating methods that runs at lower level.

Having a KLAC system can also prevent cheat developers debugging or analyzing the game, by creating extra barriers in form of restricted access to game memory, detection of unauthorized action, or implementing specific debugging mechanisms in the KLAC system [54]. This means that it will be harder and more costly for cheat developers to produce effective cheating systems, which would slow down the process of deploying and adapting cheating software to new patches for the game.

Another advantage of implementing a strong anti-cheat system on users devices is the potential mitigation of server load, which could be essential for fast-paced games, such as FPS games [54]. By implementing a KLAC system, game developers could reduce the number of cheat detection methods on the server and let the checks be handled by the local device instead. However, it is essential to note that implementing a KLAC system does not mean that the client can be fully trusted, as there still exist methods to bypass the system, such as hardware DMA's.

2.6.2 The Downsides of Kernel-Level Anti-Cheat Systems

While kernel-level anti-cheat (KLAC) improves the capability to detect low-level manipulation, it also introduces risks. The privilege of kernel drivers increases the likelihood of bugs, misconfigurations, and conflicts. In practice, the value of KLAC relies on how well stability, security, privacy, and user acceptance is managed.

Stability

Kernel drivers have the potential to fail catastrophically. A single faulty update or incompatibility with existing kernel modules can trigger Blue Screen of Death (BSOD) events, cause data loss, or prevent users from accessing systems. This was demonstrated during the global outage triggered by a faulty update to CrowdStrikes [14] Falcon sensor, a kernel-level security agent. According to Alsowaigh [3], the issue was caused by a mismatch in parameters between the sensor code and the Inter-Process Communication (IPC) Template Type, resulting in an out-of-bounds memory access within the kernel module. Because the software operated at the highest privilege level, this bug caused widespread system crashes across critical industries, including aviation, banking, and healthcare, revealing how kernel-level failures can escalate into infrastructure-wide disruptions.

Kernel-level software also presents a risk of instability due to conflicts with other privileged programs. Because such drivers interact directly with system memory and hardware resources, conflicts can occur when multiple programs attempt to control the same regions of memory or enforce overlapping protections. In the case of Battlefield 6s Javelin anti-cheat [53] and Riot Games Vanguard [39], both systems used similar memory monitoring techniques. As described by Riots anti-cheat lead Phillip Koskinas, the two drivers race to protect regions of game memory with the same technique, [34], which results in a Security Violation error when players attempt to launch Battlefield 6 while Vanguard is active. Additionally, Vanguard requires a full system reboot after being closed, which negatively impacts the user experience and interrupts normal system operation[83]. Moreover, these risks extend beyond direct resource conflicts, as kernel-level software may destabilize systems by aggressively monitoring other programs and mistakenly classifying legitimate software as malicious. The resulting issues have been reported to range from minor problems, such as mice and keyboards not functioning, or applications like Notepad failing to

open, to severe issues of device drivers used for cooling pc components being blocked and causing overheating [79].

Security

A more severe scenario occurs when a KLAC driver is compromised or impersonated. Under such circumstances, an attacker gains persistent and privileged access to process memory, input events, and critical system state. The case of Genshin Impact's anti-cheat mhyprot2.sys further illustrates this danger. [16] Because the driver was signed and trusted by the Windows kernel, it could be exploited to terminate security processes and run unauthorized code at the kernel level. The vulnerability [50] stemmed from weak access control in the Input/Output Control (IOCTL) interface [62], which failed to validate whether the calling process had sufficient privileges. Consequently, any unprivileged process could issue kernel-level commands, enabling attackers to bypass defenses entirely and gain full control over the operating system by running arbitrary code with SYSTEM privileges.

User Acceptance and Privacy

These technical risks are followed by growing distrust from users who are increasingly aware of the implications of kernel-level access. On platforms such as Reddit, players have expressed strong opposition to anti-cheat tools operating at this level. One user, SkillsBeyond, states, "...No way you should ever have access to my PC on that level..." [1], highlighting the fear of deep system penetration and potential privacy violations. Another user, AliveandDrive, questioned the extent of this control by asking, "...My question is, what can Vanguard do to my phone?..." [1]. These reactions demonstrate public concern over transparency, data handling, and the possibility of abuse. Such sentiment indicates that user acceptance is not merely a secondary consideration, but a determining factor in the long-term viability of kernel-level anti-cheat technologies.

2.7 eBPF

While the previously mentioned separation between user space and kernel space provides safety and stability (see section 2.5), it also creates limitations for applications that need deeper insight into how the system behaves. Traditionally, gaining this visibility required writing custom kernel modules or using heavyweight debugging tools. However, modern Linux systems have introduced a flexible solution for extending kernel functionality called eBPF.

eBPF is the extended version of the Berkeley Packet Filter. It provides developers with a way to extend the functionality of the kernel without loading kernel modules or modifying kernel source code, and instead relies on a virtual machine that executes sandboxed programs with kernel-level privileges [20]. eBPF programs are event driven and run whenever the kernel or an application reaches a hook point. The kernel already offers many predefined hooks, for example for system calls or the entry and exit of functions, and developers can attach eBPF programs to almost any location in the kernel or in user applications by using kernel probes or user probes. This makes eBPF a flexible mechanism for observing

or influencing system behaviour at many different levels.

An eBPF program is usually written in a restricted subset of C and compiled into eBPF bytecode [20]. Before the program can run, the eBPF bytecode must pass through the eBPF verifier. The verifier guarantees safety by checking that the program does not access invalid memory, that all control flow is bounded, and that the program will always terminate. It also ensures that the program interacts with the kernel only through approved helper functions. These checks prevent faulty eBPF programs from damaging the kernel, which is why eBPF is considered a safer alternative to traditional kernel modules. When the verifier approves the program, the kernel either interprets the bytecode or it is turned into native machine code using a Just-In-Time (JIT) compiler. The JIT compilation makes it so eBPF programs run almost as effectively as regular kernel code.

To maintain state across events and to communicate with user-space applications, eBPF programs use data structures known as maps [20]. Maps can be hash tables, arrays, ring buffers or other specialized forms of storage. They allow eBPF programs to accumulate information over time or to exchange data with user-space components, which is essential for most practical uses of eBPF. eBPF programs commonly follow an architecture and flow as the one shown in figure 2.3.

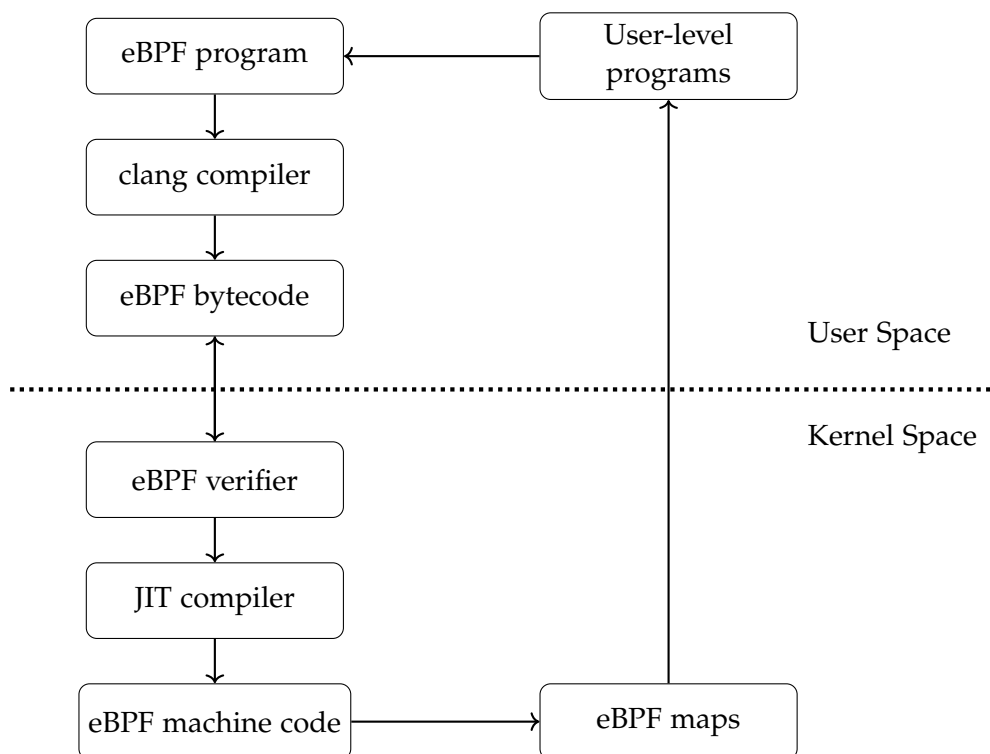


Figure 2.3: Common architecture of an eBPF program

In addition to maps, eBPF programs rely on a set of helper functions provided by the kernel. These helpers form a controlled interface that defines what an eBPF program is allowed to do, such as reading kernel data, interacting with maps, manipulating packet headers or emitting events to user space [20]. Because eBPF programs cannot access arbitrary kernel memory or call internal kernel functions directly, the helpers play an important role in maintaining safety while still giving developers useful capabilities. The specific helpers that a program may call depend on the type of hook it is attached to, which ensures that each program operates only within the context for which it was designed.

2.7.1 The Possibilities and Limitations of eBPF

eBPF's combination of event driven execution, safe extensibility and efficient data sharing between user space and the kernel has enabled a broad range of possible applications. In the area of observability, eBPF makes it possible to trace system calls, profile applications and collect detailed performance metrics without modifying the observed applications and with very low overhead [13]. In networking, eBPF supports high-performance packet processing and can implement load balancers, firewalls and routing logic directly in the kernel. In security, eBPF programs can monitor system activity in real time and enforce security policies or detect unusual behavior. Because eBPF programs can be added or removed at runtime, these capabilities can be deployed without recompiling or restarting the kernel.

Although eBPF provides a powerful and flexible framework, it also comes with several important limitations that shape how it can be used. All eBPF programs must pass the kernel's verifier, which enforces strict constraints on control flow, memory access and program size [77]. As a result, complex algorithms, large data structures or operations that require unbounded loops cannot be implemented directly in eBPF. The available functionality is further restricted by the helper functions, and if a needed capability is not provided through these helpers, it cannot be accessed from an eBPF program. Because the behavior of eBPF depends on the kernel version and configuration, portability across different systems is not always guaranteed, and older kernels may lack important program types or features.

2.7.2 CO-RE

As it was previously mentioned in section 2.7.1 one of the problems when working with eBPF has been kernel version compatibility. Since eBPF programs interact closely with kernel data structures, even small changes between kernel releases can cause an eBPF program compiled on one system to break on another [77]. This created a need to either recompile the program for each specific target kernel or rely on workarounds. Therefore eBPF introduced Compile Once - Run Everywhere (CO-RE). CO-RE enables building eBPF programs that automatically adapt to differences in kernel structure layouts at runtime, meaning a single compiled binary can run on multiple kernel versions without modification. It first of all uses the BPF Type Format (BTF), which is debugging metadata embedded in the kernel that describes the layout of structures, types, and fields. CO-RE-enabled programs read this metadata to understand the environment they are running in, rather than relying on assumptions. Instead of using fixed offsets for kernel structures, the compiler generates relocation hooks that are adjusted automatically when the program is loaded, allowing the loader to rewrite the program to match the actual field offsets of the target kernel. The userspace library libbpf performs these relocations at load time, making the entire process transparent to the developer and ensuring that a single compiled eBPF program can run reliably on multiple kernel versions.

Chapter 3

Problem Statement

Kernel modules are one of the methods for implementing low-level security monitoring solutions on Linux. This is because kernel modules give developers great visibility and control over the operating system, enabling them to intercept system calls, monitor process activity, and detect unauthorized modifications. But kernel modules come with drawbacks: bugs can lead to kernel panics and security vulnerabilities, and they require constant maintenance to maintain compatibility across kernel versions. Therefore, developers have begun migrating some security tools to eBPF instead.

eBPF allows small sandboxed programs to run at kernel level without many of the drawbacks of kernel modules. Programs are verified before execution, and only safe operations are allowed, thus preventing kernel crashes and security issues. This verification process requires the programs to be limited in complexity and features, but recent trends and papers have shown that eBPF can still, in many cases, replace kernel modules for security monitoring and runtime observability in server environments. With sources indicating a possible user concern about kernel modules being too intrusive (see section 2.6.2), it could be relevant to explore if creating an eBPF solution for anti-cheat would create higher user acceptance by being less intrusive. However, this will not be the scope of the project.

Based on this, the following problem statement was proposed:

Can eBPF programs effectively replace kernel modules for anti-cheat on Linux?

To make an effective replacement, we aim for our eBPF-solution to be more stable and create less interference with other parts of the system like the case has been with the previously presented kernel-solutions. Along with this we would have to address the following sub-questions to see if our solution would have what it takes to be a replacement for kernel modules:

- Can eBPF hooks provide the same level of system visibility as kernel modules?
- Are there any anti-cheat use cases that eBPF cannot support?
- How difficult is it for cheaters to interfere with or bypass eBPF-based anti-cheat?

Chapter 4

Methodology

In this chapter, we will cover the different methods used for certain aspects of the project. This covers both general project management, but will also cover more specific aspects of the project such as establishing requirements for our research goal, implementation of a practical solution and the evaluation of that solution.

This project is both a survey of existing research to cover the relevant methods used to attack games and an implementation of a practical solution that can be tested and evaluated. For the survey part the project revolves around a topic that is in many aspects a closed topic for the public. This means that the sources used is not only academic papers but has also been grounded in online forum post and other not academic information from the gaming industry. This has required an establishment of a certain criteria for deeming a source creditable. Therefore some knowledge have, to a certain degree, been made as an assumption of the current approaches from anti-cheat and cheating programs.

The main problem statement this paper seeks to explore is how eBPF programs can be used to effectively replace kernel modules in an anti-cheat programs. For the development and testing of this question we found it necessary to define requirements, for what it would take for a eBPF program to effectively replace kernel modules. The requirement originates from the cheating methods covered in the ADTree to define what cheating methods kernel modules are used to countermeasure. Based on this list of requirements and the time constraint of the project, some of the basic requirements were implemented into a base model for our eBPF program. This implementation was then evaluated using functional testing. The evaluation was done with the open-source game AssaultCube and used different utilities to simulate cheating behavior, such as the debugger tools Scanmem/Gameconquerer and GNU debugger GDB, as well as an open-source cheat program developed specifically for AssaultCube. This evaluation was used as a groundwork for discussing the eBPF program in the context of the research question and set a plan for future work needed.

Chapter 5

Requirements

In this chapter we will specify the requirements that a possible solution would need to effectively replace kernel modules in a KLAC system. These requirements are based on the problem analysis. We will examine how selected cheating methods work and how eBPF can be used for prevention.

5.1 Loadable Kernel Modules

Loadable kernel modules are ordinary kernel modules that contain at least an `init` function(called when the module is loaded) and usually an `exit` function(called when module is unloaded) [43]. Modules are compiled into `.ko` (kernel object) files.

To load a kernel module, tools such as "insmod" or "modprobe" can be used [6]. This operation requires root privileges, because the calling process must have `CAP_SYS_MODULE` capability which controls loading/unloading kernel modules.

Loading kernel modules requires elevated privileges, attackers who obtain root permissions will often attempt to load malicious modules. Hardening the system with restrictions for module loading can help to prevent malicious modules from being loaded [68].

Module loading can be restricted in multiple ways:

1. Kernel flag `kernel.modules_disabled=1` can block all module load/unload operations [8].
2. Enabling Secure Boot ensures that the bootloader and kernel image are signed with a trusted key, so the system boots in a trusted state. On many distributions, Secure Boot also instructs the kernel to enable lockdown mode and enforce module signing by default[76].
3. Module signing enforcement can be configured manually on distributions where it is not enabled by default. It can be turned on by booting the kernel with the `module.sig_enforce=1` parameter [7]. This ensures that only signed modules with a trusted key can be loaded.
4. Kernel lockdown can also be configured manually on distributions where it is not enabled by default. It can be turned on by booting the kernel with a `lockdown=integrity` or `lockdown=confidentiality` parameter. In lockdown mode, the kernel treats cer-

tain operations as too sensitive even for root privileges, especially if they change the integrity of the running kernel. This includes loading unsigned modules, disabling module-signing enforcement, or using interfaces that could modify the kernel's memory. With lockdown enabled, even an attacker with root capability can no longer bypass signature policy and load arbitrary modules [38].

The capability to load and unload kernel modules using `CAP_SYS_MODULE` is a security risk, and kernel features can significantly harden the module loading process [56]. In addition to these preventive measures, monitoring can be used to detect suspicious modules at runtime, for example by checking `/proc/modules` to see which modules are currently present and usable by the kernel [68]. This can help to detect unexpected or unsigned modules, especially if Secure Boot or Lockdown have been bypassed. However, a more sophisticated rootkit can try to evade this type of monitoring by deleting a corresponding entry from kernel internal linked list of all modules, which is what `/proc/modules` is generated from. Some rootkits additionally remove or hide the kernel object reference of the module under `/sys/module`. So such monitoring should be seen as an additional layer of protection, while the primary defense is to prevent untrusted modules from being loaded.

To monitor and detect suspicious modules at runtime, eBPF programs can be used. We can attach them to the `init_module` and `finit_module` syscalls to log which processes are trying to load modules and collect data about them. We can also check whether the modules have been properly signed. eBPF can hook where the kernel adds or removes entries from the module list, this way we can collect and maintain our independent view of kernel modules and compare to what `/proc/modules` exposes. If something is missing, suspicious behaviour could be flagged. While this does not replace preventive control features of kernel, it provides an additional security layer that can help to reveal suspicious module activity at runtime.

5.2 File Manipulation

Virtual File System (VFS) is a kernel abstraction layer on Linux, it provides a common interface for system calls such as `open()`, `read()`, `write()` regardless of the on-disk filesystem used (e.g. `ext4`, `xfs`) [70]. Because VFS is bridged between user-space and filesystem implementations, an attacker can tamper with user observable file contents or directory listings, without modifying data on disk. This could affect tools like `ls`, a rootkit may return filtered results, so certain files are concealed and never appear in `ls` output [68].

Virtual file systems such as `procfs` or `sysfs` enable pseudo-files that represent kernel state instead of persistent storage [68]. Reading pseudo-file calls kernel handler functions, so the output is generated dynamically. Rootkits can similarly conceal evidence by filtering the output of these handlers, so monitoring tools that rely on `/proc` and `/sys` will see tampered outputs. For example, `/proc/modules` lists all currently loaded LKMs and `/sys/module` exposes information about loaded modules. Both can be manipulated to hide a malicious module from user-space.

This attack is often used early to remain hidden as user space provides the system's state information through interfaces and if any of those interfaces are manipulated, many tools will become unreliable. Since this rootkit requires kernel-level code execution, preventing

loading of malicious LKMs and monitoring suspicious modules using eBPF can prevent such attacks, which is covered in section 5.1.

Additionally, eBPF can attach to LSM hooks to enforce that only specific trusted processes can write to `/proc` and `/sys`. Using kprobes we can also trace load/unload attempts to detect and analyze which process and file is being used to load the module.

5.3 File Integrity

File integrity checking can help to detect malicious changes made to files. Setting up a known baseline of files by creating cryptographic hashes for them and checking hashes periodically can help to confirm whether files have been modified.

Many integrity systems create integrity data before being compromised such as rkhunter [61]. For each file it stores a known set of properties like cryptographic hash, permissions and ownership. If file hash or other attributes differ from the baseline, rkhunter flags them. Such attacks targeting integrity can be used for systems to behave differently than intended. For example, modify an applications logic.

In addition to scanning which typically happens periodically, monitoring can be used to detect file modifications at runtime, so suspicious behaviour could be detected earlier than scheduled scans. eBPF programs can be used to monitor integrity-impacting events. They can attach to syscalls such as `openat`, `write`, `rename`, `unlink`, `chmod` and `setxattr` to monitor processes that are accessing or modifying files. If sensitive files are modified by unexpected processes, suspicious behaviour can be flagged. This provides a layer of protection to detect tampering attempts.

5.4 Memory Scanning

Wallhacks, aimhacks, and triggerbots need access to the games memory to work. The memory stores useful information about the current game state such as player location, enemy location, health, etc. Cheating programs read this information and use it to gain an unfair advantage: wallhacks use the enemy location to draw enemies through walls, aimbots use the enemy location and player view angle to calculate mouse movement to instantly aim at the enemy, and triggerbots check if the player is aiming at an enemy and automatically fires. A major challenge for cheat developers is finding the memory address that holds a specific value(for example, an enemies location). Memory scanning tools like GameConqueror and CheatEngine can be used to scan the entire memory for a specific known value to locate it. However, this value is most likely stored in dynamic memory, so its address changes every time the game restarts. Therefore, once a dynamic address is found, the cheat developer must search for a pointer to that address. This pointer might also not be static, so the process is repeated until a chain of pointers is found that starts at a static memory address. This chain of pointers forms a memory access graph and by traversing this (static base pointer -> pointer 1 -> pointer 2 -> enemy location), the cheat can reliably find the target value every time the game runs [27].

On Linux, memory access between processes can be categorized into user-level and kernel-level methods.

5.4.1 User-Level Memory Access

User-level cheats use standard system calls, which are intended for debugging or inter process communication purposes.

The `ptrace` syscall allows one process to observe and control another process [45]. After attaching to another process, the tracer can read and write memory and registers, set breakpoints, and stop and resume execution (note: The `ptrace` syscall requires root privileges or re-configurations to attach to and control another process):

```
ptrace(PTRACE_ATTACH, pid, NULL, NULL);
long data = ptrace(PTRACE_PEEKDATA, pid, address, NULL);
```

The `/proc` filesystem provides a simple file-based interface to various data structures. For process memory access, the file `/proc/pid/mem` allows reading and writing of a process memory. You can use standard file operations for this, such as `open`(and its variations), `lseek`, `read`, and `write`:

```
int fd = open("/proc/pid/mem", O_RDONLY);
lseek(fd, address, SEEK_SET);
read(fd, dst_buffer, length);
```

Furthermore, syscalls `process_vm_readv` and `process_vm_writev` can also be used for malicious purposes because they are designed for data transfer between two processes.

Detection

Assuming the cheat software runs at user-level and uses the syscalls mentioned previously in this section, a strategy could be to use eBPF to place hooks on those system calls to detect access to the protected game process. Reading and writing memory requires a file descriptor (FD) to that process. Blocking the function used to create FDs could potentially block access to reading and writing memory. However, this has to be tested to be confirmed.

Therefore, the list of syscalls that should be monitored is:

- `ptrace`
- `open`
- `openat`
- `openat2`
- `process_vm_readv`
- `process_vm_writev`

Beyond syscall monitoring, developers can increase the time and complexity of cheat development through pointer encryption. By encrypting or otherwise obfuscating pointers to sensitive memory and only decrypting them immediately before use, you break the "pointer chain". Cheat developers will eventually reverse-engineer the decryption procedure, so this is not a permanent solution but it does increase the time required to update a cheat after a game patch [27]. However, since this feature must be uniquely implemented in every game, it will not be part of our solution.

5.4.2 Kernel-Level Memory Access

User-level monitoring is a good first step, but it is insufficient for cheats that use custom kernel modules. A kernel-level cheat can bypass syscall hooks entirely by directly calling kernel-level functions. The open source Github project by IamSanjid [28] is an example of such technique. It was built to read and write memory of a process through a custom kernel module, and we can use this repository to find functions that are relevant for an anti-cheat to monitor by looking at the source code of the kernel module.

We found that `pid_task` and `find_vpid` are the "entry point" for many of the potential malicious operations. Together, these functions are used to resolve a `task_struct` from a process id (PID). The `task_struct` contains a lot of information about a specific process that is useful for cheat developers. For example, it includes the start and end addresses of the executable code segments, the heap, and the stack. Once the `task_struct` is obtained, the module can use `get_user_pages_remote` (or similar "gup" functions [55]) to gain direct access to user-space memory. This function ensures that the target memory pages are pinned in RAM and cannot be swapped to disk while the module is accessing them, providing a safe way for reading or overwriting game data.

Detection

To counter kernel-level memory access, we can use eBPF kprobe programs. These allow the anti-cheat to attach monitoring logic to the entry and exit points of all exported kernel functions. A full-scale implementation would need to monitor a lot of different kernel functions but for this proof of concept however, we will just focus on the core functions of the kernel module mentioned above:

- `pid_task`
- `find_vpid`

5.5 Virtual Machine

As identified in our ADTree (see figure 2.1) and described in paper [32], virtual machines (VMs) can be used to bypass anti-cheat systems. In this attack model, the game and its anti-cheat software execute inside a guest operating system (the OS running inside the VM), while cheat-related logic operates outside the guest operating system. This separation limits the visibility of the anti-cheat and complicates monitoring techniques.

According to IBM’s documentation on virtualization [67], virtualization introduces a hypervisor layer between the guest operating system and the physical hardware. The hypervisor (also known as a virtual machine monitor) is responsible for managing and isolating multiple guest operating systems, mediating access to CPU execution, memory management, and I/O devices.

As a result, assumptions commonly relied upon by anti-cheat systems, such as trusted kernel execution, direct hardware access, and consistent timing behavior, may no longer hold. Certain operations, such as memory inspection, debugging, and execution monitoring, can be performed externally by the hypervisor without invoking any observable system calls inside the guest operating system.

As described in the paper [32], this represents a significant shift from user-level or kernel-level cheats. While those traditional approaches rely on observable system interactions, VM-based cheats interact with the guest from the outside. Because these interactions do not trigger standard system signals or API calls, the anti-cheat remains unaware of any direct memory access or debugging activity occurring at the hardware level.

Detection

The papers [10], [25] highlight VM detection mechanisms, that anti-cheat systems also can use, to determine whether a game is running inside a virtual machine:

One approach is to examine hardware-level properties: virtual machines can exhibit differences in CPU feature responses, instruction behavior, or execution timing due to the way hypervisors schedule and manage instructions.

Detection can also rely on low-level artifacts caused by the hypervisor, such as subtle changes in microarchitectural states, cache behavior, or timing patterns, which can be measured from within the guest system to indicate virtualization.

Behavior-based indicators can additionally be used. Interactions with hardware, instruction sequences, or performance counters may behave differently in a VM, and these deviations can signal the presence of a hypervisor.

Using a combination of these methods would be required for detection, as each approach alone can often be bypassed or masked.

5.6 Secure Boot

Secure Boot is a security feature designed to ensure that a system only boots software that is cryptographically verified[80]. This is especially important in environments that depend on strict integrity enforcement. Modern cheats abuse custom or vulnerable kernel drivers to gain kernel-level privileges and better disguise their presence from anti-cheat systems[18]. Without a trusted boot path, an attacker can replace the operating system kernel or load a custom hypervisor before the OS initializes, compromising the system’s security and effectiveness of anti-cheats.

TPM

A Trusted Platform Module (TPM) is a security component built in hardware, usually as a discrete chip on the motherboard. It is designed to provide built in hardware cryptographic operations such as key generation, hashing, device attestation, and secure storage of secrets [48]. Unlike software-based solutions, TPM is isolated from the operating system, making it resistant to tampering.

One of its primary use cases is to establish a measured boot process. During system startup, each component in the boot chain generates a cryptographic hash of its code and configuration. This hash is sent to the TPM and "extended" into a Platform Configuration Register (PCR) [80]. An extend operation is a one-way function: $\text{PCR}_{\text{new}} = \text{HASH}(\text{PCR}_{\text{old}} || \text{new_hash})$. This creates an immutable log of the systems boot process that can later be used to verify system integrity.

Chain of Trust

At a high level, Secure Boot establishes a chain of trust [82]. The system firmware (UEFI) contains trusted public keys that validate the first boot component. Each stage in the boot sequence must validate the subsequent one: the firmware verifies the bootloader, which in turn verifies the kernel and its associated modules. This sequential validation process, illustrated in Figure 5.1, ensures that each link is signed by a key rooted in the firmwares trust vault. Because of this, an attacker cannot introduce a modified or unsigned component. This establishment prevents attackers from loading tampered kernels, injecting early-boot rootkits, or bypassing kernel-mode protections that anti-cheat systems rely on.

Secure Boot on Linux

In Linux, Secure Boot[76] is implemented using a multi-layered mechanism built around the shim bootloader [57]. Shim is a small, signed intermediary that allows Linux to participate in UEFI Secure Boot without relying solely on OEM-embedded keys. Shim validates the bootloader, which in turn validates the Linux kernel image. Linux then enforces module-signing: all kernel modules must be signed with a key trusted by the kernels keyring or enrolled via the Machine Owner Key (MOK) system [40].

Many Linux distributions also enable kernel lockdown mode by default if secure boot is enabled. A combination of both provides a more secure environment as secure boot protects the OS boot process and lockdown protects the kernel once its running [38]. Kernel lockdown helps restrict modifications to the running kernel by enforcing signature verification for kernel modules and limiting interfaces that directly access kernels memory. The lockdown feature can also be enabled by a user to run in either integrity or confidentiality mode. In integrity mode, new kernel modules must be cryptographically verified, while confidentiality mode also restricts access to sensitive kernel data. Lockdown offers protection once the operating system is running, however, it does not guarantee that kernel is trustworthy as it does not guarantee that authorized modules are loaded into expected kernel. Establishing kernel trust requires combining Secure Boot, Kernel Lockdown and Measured Boot should be additionally added for remote attestation of the boot chain and applications.

Together, these mechanisms strengthen security of both the boot chain as shown in Figure 5.1 and the running kernel on Linux systems. By ensuring that the system boots into an untampered kernel and that only verified modules can be loaded [38], Secure Boot provides a foundational guarantee that higher-level anti-cheat mechanisms can trust the underlying platform [15]. Although Secure Boot alone cannot address every attack, it helps to prevent modifications to the bootloader, operating system and application code, making it a critical baseline security component.

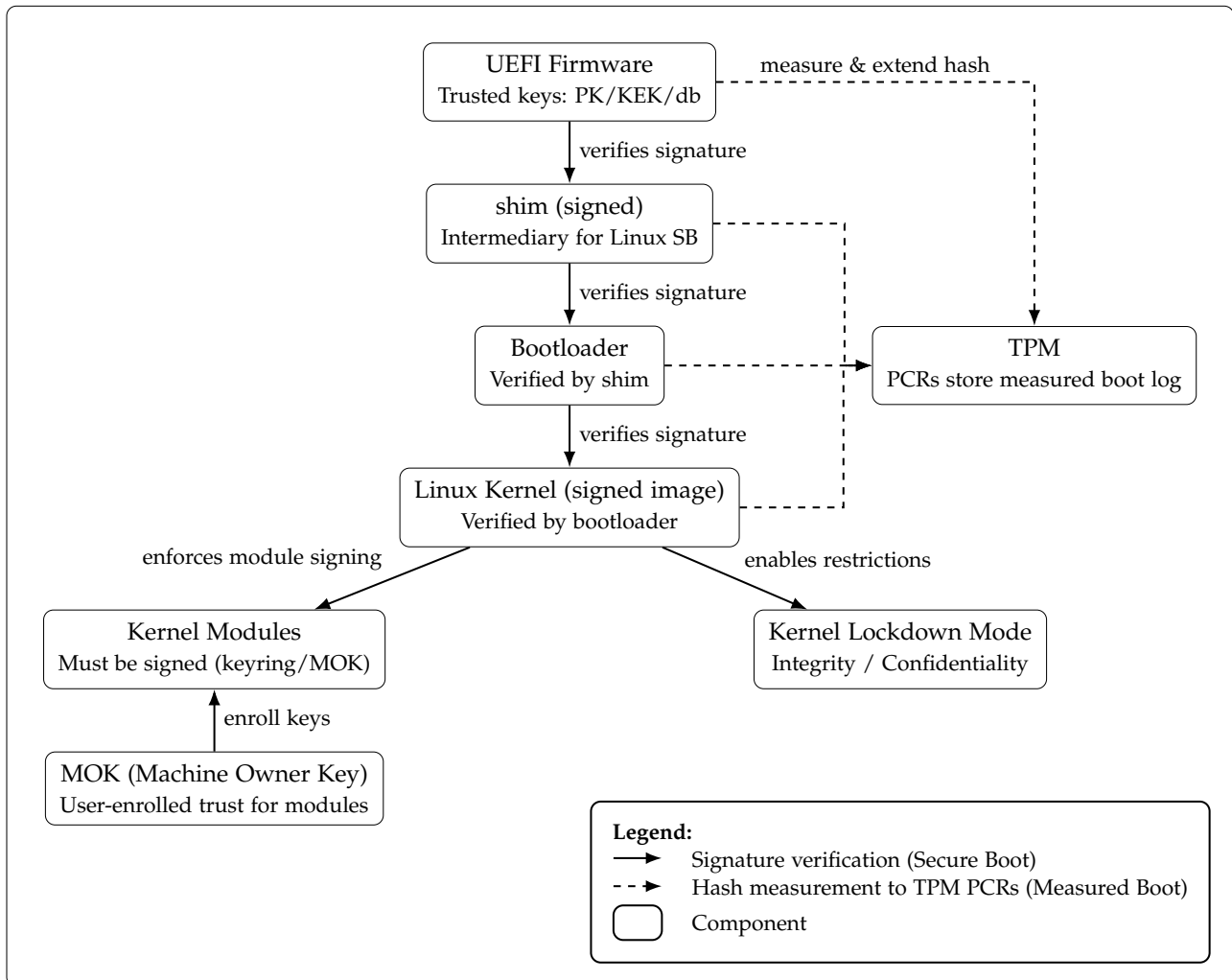


Figure 5.1: Secure Boot on Linux: Chain of Trust.

5.7 Summary of Requirements

To sum up, the following was what we deemed necessary for an eBPF solution to effectively replace kernel modules:

1. Detect suspicious kernel modules being loaded at runtime.
2. Combat file manipulation to minimize the possibility of hiding malicious modules.
3. Along with the previous, file integrity checks to detect malicious changes.
4. Memory scanning measures to combat common cheats like wallhacking and aimhacking that utilize game memory to work. This must be done on both User-level and Kernel-level.
5. Detection of virtual machines, to combat cheaters using it to separate where the anti-cheat and cheat are being executed.
6. Secure Boot implementation to establish a chain of trust.

Chapter 6

Implementation

As part of this project, we developed TyrSecure, an anti-cheat software solution designed to monitor the operating system for suspicious events using eBPF. This chapter breaks down TyrSecures architecture and implementation.

The architecture is divided into two modules:

- Memory Access Tracker: Provides real-time protection for a running application's memory space.
- Kernel Module Tracker: The unfinished feature that monitors the loading and unloading of kernel modules.

The operational flow of these modules follows an event-driven design, as illustrated by the Memory Access Tracker module in Figure 6.1. The system is divided between user-space and kernel-space execution.

In the user-space layer, the Memory Agent initializes the Memory Handler and registers a callback function to manage incoming security events. Furthermore, the Memory Agent implements a logging utility to format and visualize the data captured from the kernel. The Memory Handler serves as the interface to the eBPF programs. Its primary responsibilities are managing the lifecycle of the eBPF programs and communication by initializing a ring buffer that can be written to and read from by both eBPF programs and user-space programs.

In kernel-space, the Memory Tracker implements its monitoring logic by attaching to pre-defined hook points (such as tracepoints, LSM hooks, and kprobes). These hooks act as interceptors at execution paths within the kernel. When triggered, the logic executes and transmits the captured events to user-space via the ring buffer. The implementations are explained in the following sections.

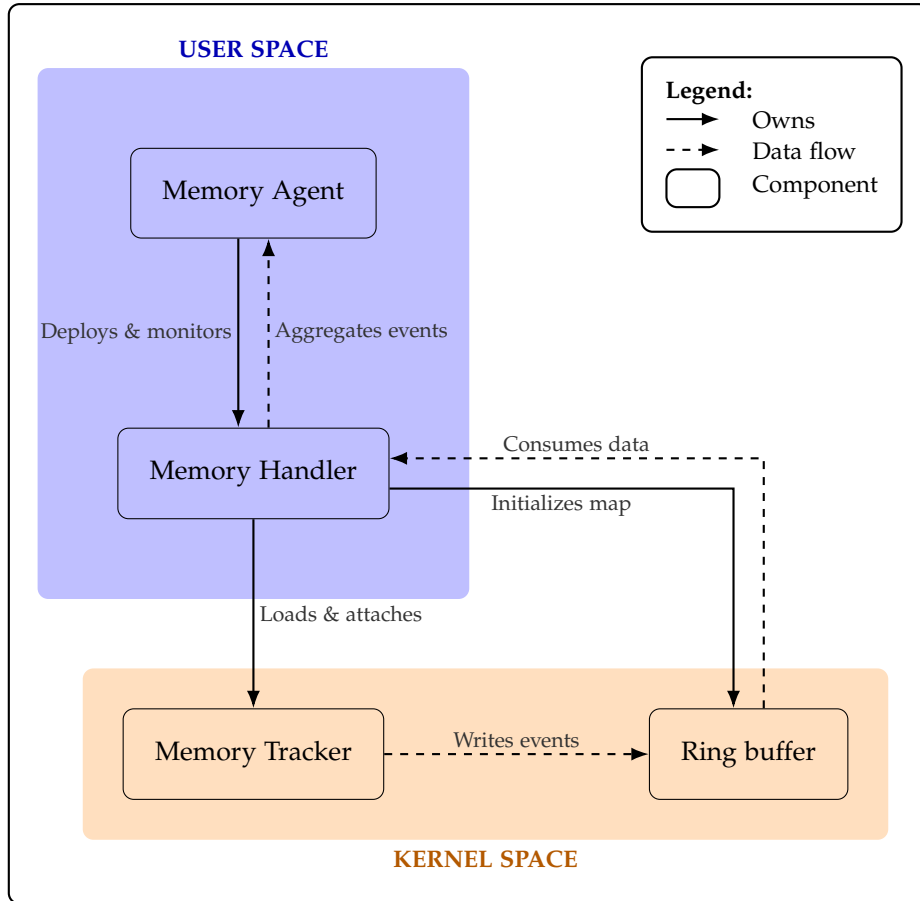


Figure 6.1: Structural overview of the memory access module.

6.1 The Memory Agent

The Memory Agent component is split into these features: a constructor to set it up in the right state, an `on_event` callback to delegate logic to the handler and `get_next_event` to fetch data from its queue data structure. The constructor (lines 2-6) establishes the primary communication bridge. It initializes the handler with a lambda callback, which instructs the system to route every kernel-level memory event directly to the agent's `on_event_cb` method. After assigning the `protected_pid` to the class member, it invokes `handler.LoadAndAttachAll`. This final step pushes the tracking configuration into the kernel, targeting the specific process ID for monitoring.

The `on_event_cb` function (lines 10-13) is the entry point for all incoming memory events. Right now, the event data is just stored in a queue, but in the future, this is where analysis and additional filtering will happen. It utilizes `std::lock_guard<std::mutex>` to synchronize access to the internal storage queue; this effectively "locks the door" to the queue, preventing the background kernel thread and the user-facing application from modifying the data simultaneously. Once the lock is held, the event is pushed onto the `event_queue`, buffering it for analysis.

The `get_next_event` function (lines 15-22) serves as the interface for the application to consume the collected data. Like the callback, it first secures the `queue_mutex` to ensure thread-safe retrieval. It then evaluates the state of the buffer: if the queue is empty, it

returns `std::nullopt`, providing a clean way for the application to handle periods of inactivity. If data is present, the function captures the oldest event using `front()`, removes it from the queue with `pop()`, and returns the event to the caller for processing.

```
1 ...
2 mem_access_agent::mem_access_agent(pid_t protected_pid)
3     : handler([this](const mem_event &e) { on_event_cb(e); }) {
4     this->protected_pid = protected_pid;
5     handler.LoadAndAttachAll(protected_pid);
6 }
7
8 void mem_access_agent::on_event_cb(const mem_event &e) {
9     std::lock_guard<std::mutex> lock(queue_mutex);
10    event_queue.push(e);
11 }
12
13 std::optional<mem_event> mem_access_agent::get_next_event() {
14     std::lock_guard<std::mutex> lock(queue_mutex);
15     if (event_queue.empty())
16         return std::nullopt;
17     mem_event e = event_queue.front();
18     event_queue.pop();
19     return e;
20 }
21 ...
22 }
```

Figure 6.2: The Memory Agent's code.

6.2 The Memory Handler

The `ring_buffer_callback` (lines 1-15) is the function that processes raw data directly from the kernel's memory. It first validates the incoming data size against `sizeof(mem_event)` to ensure the memory has not been corrupted. After validation, it performs a `std::memcpy` to translate the raw kernel bytes into a structured C++ object and passes it to the agent via the `on_event` callback.

```
1 ...
2 int mem_access_handler::ring_buffer_callback(void *ctx, void *data,
3                                              size_t data_sz) {
4     if (data_sz != sizeof(mem_event)) {
5         return 1; // Return non-zero to indicate a processing error
6     }
7
8     auto *handler = static_cast<mem_access_handler *>(ctx);
9
10    mem_event e;
11    std::memcpy(&e, data, sizeof(e));
12    handler->on_event(e);
13
14    return 0;
15 }
```

Figure 6.3: The Memory Handler callback code.

The `LoadAndAttachAll` function 6.4 orchestrates the setup of the eBPF programs. It begins by opening the BPF skeleton, which serves as the blueprint for the kernel code. It then injects the `protected_pid` into the skeleton's read-only data, this tells the kernel exactly which process to watch. Once configured, it loads the program into the kernel and creates a Ring Buffer, a specialized memory area where the kernel writes events and the user space reads them.

Finally, the function spawns a background thread using `std::jthread` (lines 24-30). This thread runs a continuous loop that calls `ring_buffer__poll`, which waits for the kernel to send new data. This design ensures that the monitoring is non-blocking; the main application remains responsive while the handler background thread handles the heavy lifting of listening for kernel events.

```

1
2 int mem_access_handler::LoadAndAttachAll(pid_t protected_pid) {
3
4     if (!on_event) return -1;
5
6     if (!(skel_obj.reset(mem_access__open()), skel_obj)) return -1;
7
8     skel_obj.get()->rodata->PROTECTED_PID = protected_pid;
9
10    if (mem_access__load(skel_obj.get());) {
11        skel_obj.reset();
12        return -1;
13    }
14
15    rb.reset(ring_buffer__new(bpf_map__fd(skel_obj->maps.rb),
16        mem_access_handler::ring_buffer_callback, this, nullptr));
17
18    if (!rb || mem_access__attach(skel_obj.get())) {
19        rb.reset();
20        skel_obj.reset();
21        return -1;
22    }
23
24    loop_thread = std::jthread([this](std::stop_token st) {
25        while (!st.stop_requested()) {
26            if (ring_buffer__poll(rb.get(), 100) < 0 && errno != EINTR) break;
27        }
28    });
29    return 0;
30 }
31 ...
32 }

```

Figure 6.4: The Memory Handler load code.

6.3 The Memory Tracker

The Memory Tracker serves as the monitoring logic of the system, responsible for observing kernel activities and enforcing access control policies in real-time. The Memory Tracker is implemented as six eBPF hooks:

1. `tp/syscalls/sys_enter_ptrace` (tracepoint)
2. `tp/syscalls/sys_enter_process_vm_readv` (tracepoint)
3. `tp/syscalls/sys_enter_process_vm_writev` (tracepoint)
4. `kprobe/find_vpid` (kernel return probe)
5. `kretprobe/pid_task` (kernel probe)
6. `lsm/file_open` (Linux Security Module)

These hooks are placed across different layers of the Linux kernel to capture suspicious events, ranging from system calls to internal function executions. Combined, they cover the requirements listed in section 5.4. The following subsections detail the implementation of the three different hook types: tracepoints, kernel probes, and the Linux Security Module (LSM) framework.

Tracepoints

Tracepoints are predefined hook points in the Linux kernel where eBPF programs attach. When the kernel reaches one of these points, the attached eBPF program is triggered. We used tracepoints to monitor most of the syscalls identified in section 5.4. An example can be seen in Figure 6.5 where the `ptrace` tracepoint is implemented to detect when `ptrace` is called on the protected process. On line 10, we use the `SEC` macro to tell the compiler in which ELF section to place the function. `Libbpf`(the eBPF loader we use) interprets this information to determine which type of eBPF program it is, and where to place the hook. In this case, it is a tracepoint(specified with `tp`), the subsystem to be monitored is `syscalls` and the specific tracepoint is `sys_enter_ptrace`.

```

1 volatile const pid_t PROTECTED_PID;
2 struct mem_event {
3     enum mem_event_type type;
4     uint32_t caller;
5     uint32_t target;
6     char caller_name[MY_TASK_COMM_LEN];
7     char filename[MY_FILENAME_LEN];
8 };
9
10 SEC("tp/syscalls/sys_enter_ptrace")
11 int ptrace_entry(struct trace_event_raw_sys_enter *ctx) {
12     pid_t caller = (pid_t)(bpf_get_current_pid_tgid() >> 32);
13     pid_t target = (pid_t)BPF_CORE_READ(ctx, args[1]);
14
15     if (target != PROTECTED_PID || caller == PROTECTED_PID)
16         return 0;
17
18     struct mem_event *e = bpf_ringbuf_reserve(&rb, sizeof(struct mem_event)
19         , 0);
20     if (!e)
21         return 0;
22
23     e->type = PTRACE;
24     e->caller = caller;
25     e->target = target;
26     bpf_get_current_comm(e->caller_name, sizeof(e->caller_name));
27
28     bpf_ringbuf_submit(e, 0);
29
30     return 0;
31 }

```

Figure 6.5: An eBPF program that detects when ptrace is called with suspicious parameters.

The second parameter of the ptrace syscall represents the PID of the tracee. We can use BPF_CORE_READ on line 13 to read that and check if it matches the protected process' PID. If the target PID does not match our protected process, the program just terminates. Otherwise, we want to send the event to user-space through the ring buffer. Line 18 allocates space by referencing the mem_event structure (lines 2-8) to reserve a memory segment. Lines 19-20 validate this allocation to prevent null-pointer dereferences and to satisfy the verifier. Finally, lines 22-25 populate the event metadata, including PID and name of the caller and the PID of the target.

Implementation of the process_vm_readv and process_vm_writev tracepoints follows nearly identical logic, differing only in the argument index used to retrieve the target PID. These are omitted for brevity.

Kernel Probes

To expand monitoring beyond predefined tracepoints in the kernel, we used kprobes(kernel probes) and kretprobes(kernel return probes). They are similar to tracepoints, except we can place them at any exported kernel function(functions that are available to loadable kernel modules). Fortunately, all the kernel functions listed in subsection 5.4.2 are exported, so it is possible to place a probe in find_vpid and a return probe in pid_task. As shown

in Figure 6.6, TyrSecure employs a kretprobe on the kernel function `pid_task`. Like the eBPF program in the previous subsection, this is specified in the SEC macro. This allows the system to monitor whenever the kernel successfully resolves a `task_struct` for a specific `PROTECTED_PID` set at the user-space handler module. If the resolved process matches the `PROTECTED_PID` seen at line 7, the eBPF program identifies this as a potential unauthorized access attempt and exports the event via the ring buffer for further inspection by the user-space agent through the handler. The implementation of the kprobe that monitors `find_vpid` it is similar and will not be shown here but can be found in the code base.

```

1  const pid_t PROTECTED_PID;
2
3  SEC("kretprobe/pid_task")
4  int BPF_KRETPROBE(kprobe_pid_task_exit, struct task_struct *return_val)
5  {
6      pid_t target_pid = BPF_CORE_READ(return_val, pid);
7      if (target_pid != PROTECTED_PID) {
8          return 0;
9      }
10
11     struct kernel_event *e = bpf_ringbuf_reserve(&rb, sizeof(struct
        mem_event), 0);
12     // submit event to ring buffer
13     ...
14 }
```

Figure 6.6: An eBPF kretprobe monitoring successful task structure resolutions for the protected process. Edited for readability.

Linux Security Module

In addition to monitoring, eBPF can also be used to actively enforce security policies with the Linux Security Module(LSM) framework. Unlike tracepoints or kprobes, LSM programs can return an error code to block suspicious actions before they are executed. As shown in Figure 6.7, we have implemented an access control policy for the `/proc` filesystem. This program hooks deep within the kernel instead of targeting a specific syscall, effectively covering all open variants that try to access the `/proc` filesystem. However, because this program runs hundreds of times per second (on our systems), performance is important to keep in mind.

Performing a full string comparison on every file would be expensive and a waste of CPU resources. Therefore an initial filtering is done on line 9-11, where all requests to non `/proc` files are allowed with a single integer comparison. Every file in the `/proc` filesystem has a magic number of `0x9fa0` as defined in the Linux header `magic.h`. To read the magic number, we use the `BPF_CORE_READ` macro, which improves portability between Linux version by including additional debug information to calculate field offsets before execution. The equivalent non CO-RE code would be `file->f_inode->i_sb->s_magic`.

Next, the program uses string comparison to detect access to restricted files (`/mem`, `/maps`, and `/smaps`). While these paths are flagged as security risks due to their use in memory inspection, we must still allow access for files like `/status` to ensure that legitimate tools works correctly. Starting on line 20, the parent directory name is now examined. Since `/proc` is structured with subdirectories corresponding to PIDs (for example

/proc/pid/maps), the code verifies if the parent directory matches the `protected_pid_s` string. If it does, at this point, we can assume that it is a suspicious action and an event is sent to the ring buffer and the action is denied by returning `-EPERM` operation not permitted.

```

1 #define PROC_SUPER_MAGIC 0x9fa0
2 SEC("lsm/file_open")
3 int BPF_PROG(restrict_proc_access, struct file *file)
4 {
5     pid_t caller_pid = bpf_get_current_pid_tgid() >> 32;
6     if (caller_pid == PROTECTED_PID)
7         return 0; // allow process to access itself
8
9     unsigned long magic = BPF_CORE_READ(file, f_inode, i_sb, s_magic);
10    if (magic != PROC_SUPER_MAGIC)
11        return 0; // return if not part of procfs
12
13    char *name = (char*)BPF_CORE_READ(file, f_path.dentry, d_name.name);
14
15    bool is_restricted_file_name = (bpf_strcmp(name, "maps") == 0 ||
16                                   bpf_strcmp(name, "smaps") == 0 ||
17                                   bpf_strcmp(name, "mem") == 0);
18    if (is_restricted_file_name)
19    {
20        char *parent_name = (char*)BPF_CORE_READ(file, f_path.dentry,
21                                                  d_parent, d_name.name);
22        char protected_pid_s[16];
23        BPF_SNPRINTF(protected_pid_s, sizeof(protected_pid_s),
24                     "%d", PROTECTED_PID);
25
26        bool is_parent_protected_pid =
27            bpf_strcmp(protected_pid_s, parent_name) == 0;
28
29        if (is_parent_protected_pid) {
30            // submit event to ring buffer
31            ...
32            return -EPERM;
33        }
34        return 0;
35    }
36
37    return 0;
38 }

```

Figure 6.7: A LSM hook that controls access to the /proc file system. Edited for readability.

6.4 Kernel Module Tracker

The Kernel Module Tracker was designed to detect the insertion of potentially malicious kernel modules that could be used to cheat. While the Memory Tracker focuses on specific process isolation, this module provides broader system-wide visibility. The Memory Tracker is implemented using a tracepoint attached to `module_load`. This tracepoint is triggered by the kernel immediately after a module has been loaded. While the eBPF implementation of the Memory Tracker is functional, its effectiveness as an anti-cheat measure is currently limited by the timing of its execution. Additionally, it only collects

information and still need more features to have an effect on cheats. Since TyrSecure only operates at runtime, the eBPF hooks are only attached once the game boots. This leaves time for the cheater to load a kernel module before TyrSecure can detect it.

6.5 Project Structure

Selecting the right tools was essential for our eBPF implementation. Since eBPF is natively written in C, we leveraged our prior experience with the language to develop the core components. To ensure the project remained maintainable and easy to compile, we used a development tool called CMake, which is a cross-platform tool designed to build, test, and package software.

The development of eBPF programs additionally requires the use of bpftool, which is a tool for managing eBPF objects. In our program bpftools is responsible for generating skeleton code from compiled eBPF .o objects. These skeletons enable user-space applications to interact with eBPF programs, as eBPF code must be loaded into the Linux kernel from user space. The user-space components were implemented in C++ using the libbpf library, which provides functionality for loading, attaching, and communicating with eBPF programs.

The project is structured around CMake conventions and organized into three main directories: src, app, and cmake. The src directory contains all core implementations, the app directory integrates these components to build the final executable, and the cmake directory contains build automation logic.

The CMake scripts within the cmake folder performs several essential tasks, including verifying the availability of required libraries, ensuring the target system is Linux-based, and triggering the generation of vmlinux.h from bpftool, which is a mandatory dependency for eBPF development.

To maintain a clean separation of concerns, the src directory is further modularized into subdirectories:

- eBPF_prog: Contains the kernel-space C code that defines the probes and tracing logic.
- common: Holds shared data structures, such as structs used in BPF maps, ensuring that both kernel and user space interpret data consistently.
- handlers: Encapsulates the logic for managing the eBPF lifecycle. This module handles the loading and unloading of programs and utilizes dedicated threads to retrieve data from BPF maps without blocking the main application logic.
- agents: Provides a high-level abstraction that aggregates data collected by the handlers, storing it in a queue-based data structure for subsequent processing.

Chapter 7

Evaluation

In this chapter we will go into how we evaluate the requirements implemented in TyrSecure, and present the evaluation result.

7.1 Test Setup

For evaluating TyrSecure we want to test its effectiveness against memory scanners and debuggers, when they are attached to a game process. This aligns with the capability we have implemented for TyrSecure, which currently includes features against memory accesses. Furthermore, we would also evaluate the current version of TyrSecure against a complete cheat program, to test its capabilities against a more complex cheating method. For the test four tools were used:

Tool name	Tool version	Tool description
AssaultCube	Version 1.2.0.2	Open-source game used for test
GDB	Version 17.1	Memory scanner and debugger tool
Scanmem	Version 0.17	Memory scanner and debugger tool
ac_rhack	4. June 2021	Open-source cheat program

Table 7.1: Tools used in test

AssaultCube was chosen due to its open-source nature, which makes it ethically appropriate to apply and test cheating methods on. This has made it a common target for developers and "hackers" to experiment with and implement various cheating techniques. For simulating a complete cheating program we used an open-source cheat ac_rhack[63]. ac_rhack is only compatible with AssaultCube version 1.2.0.2, which is also why that version is used for the test setup. ac_rhack works by using the LD_PRELOAD operation. This operation will make the cheat program load before the game process is initialized, and overrides any shared libraries that the game process uses. This means that the cheat program does not monitor or manipulate the game memory in any form and indicate that it will be out of the scope of the current capabilities of TyrSecure, which is also reflected in our expectations of the test-case defined later. The tests are performed on a VirtualBox VM running Arch Linux with kernel version 6.18.2-arch2-1. A total of seven test cases were performed: three test cases that used GDB, three test cases that used Scanmem and one test case using ac_rhack. To distinguish between the tests, the following test cases were

established with unique naming and expected outcome of the individual tests (as seen in table 7.2). For our expectations of the test cases we use the term "intentional behavior", which is used to describe the steps taken for each tool to work. Intentional behavior refer to different steps for each tool. For GDB the steps are: 1) being able to attach to game process 2) being able to access memory mappings and their stats. The steps for Scanmem are: 1) being able to attach to a game process and monitor related memory regions 2) isolate the addresses of an arbitrary variable 3) being able to change this value. Finally ac_rhack steps are 1) being able to launch game succesfully and 2) all of its eight cheating features, which includes: Wallhack, Aimbot, Autoshoot, No Recoil, No Spread, Infinite Ammo, Invincibility, 1-shot-1-kill works successfully.

Test name	Test description	Test expectations
GDB-1	Control test: Attaching GDB to the game process without TyrSecure being involved.	GDB is expected to function as intended.
GDB-2	TyrSecure before GDB: TyrSecure is attached to the game process before attaching GDB.	GDB is expected to be able to attach to the game process but not access memory mappings or their stats.
GDB-3	TyrSecure after GDB: GDB is attached to the game process before attaching TyrSecure.	GDB is expected to function as intended while TyrSecure is not attached. After TyrSecure is attached, GDB is expected to be unable to successfully access memory mappings or their stats.
SM-1	Control test: Attaching Scanmem to the game process without TyrSecure being involved.	Scanmem is expected to function as intended.
SM-2	TyrSecure before Scanmem: TyrSecure is attached to the game process before attaching Scanmem.	Scanmem is expected to be able to attach to the game process but not any further steps of its intended behavior.
SM-3	TyrSecure after Scanmem: Scanmem is attached to the game process before attaching TyrSecure.	Scanmem is expected to function as intended while TyrSecure is not attached. After TyrSecure is attached, Scanmem is expected to be unable to successfully perform any remaining steps of its intended behavior.
CP-1	Open-source cheat program: cheat is attached to game before TyrSecure is attached to the game process	Expect ac_rhack to function as intended and do not expect that the eBPF program will be able to detect the cheat program.

Table 7.2: Test cases

7.2 Test Method

All tools used have different workflows in order to achieve what we defined as intended behavior.

GDB works by attaching the tool to the PID of the game process, this action requires root privileges to do. After GDB is attached, we will try to access memory mappings. This is done by printing mappings and the stats for them from the virtual filesystem `/proc/` for the game process PID. The commands for this is `"shell cat /proc/<game pid>/maps"` or `"shell cat /proc/<game pid>/smaps"`.

Scanmem takes the following steps: **Step 1:** Use command `PID <pid>` to attach scanmem to game process. **Step 2:** Enter `<variable value>` to search for a variable with same value as looking for. **Step 3:** Change wanted value in-game. **Step 4:** Enter `<variable value>` to the new value, after changed in-game. **step 5:** Repeat step 3 and 4, until variables addresses is narrowed down sufficiently. **Step 6:** Change value of the variable to wanted using `set <value>` command.

The workflow of launching `ac_rhack` is only one command and is to use the `LD_PRELOAD` command with the path to the shared `.io` library file of the cheat program, along with the shell script for `AssualtCube`.

For all test cases these tool specific steps were taken in chronological order. This was done to see how many steps each tool was able to do successfully. If a tool was not able to do a specific step successfully the test stopped, else it stopped when all steps were done successfully. Besides the tool specific steps there were another test step that involved attaching `TyrSecure` to the game process. When this step is performed varies based on test cases: for tests `SM-1` and `GDB-1` the step is not performed, for tests `SM-2` and `GDB-2` the step is performed before any steps related to `Scanmem` and `GDB` and lastly for tests `SM-3`, `GDB-3` and `CP-1` the step is performed after `Scanmem`, `GDB` and `ac_rhack` is attached to game process.

7.3 Test Result

In this section we will go over all of the test cases and their results, and finally discuss and conclude on the overall result of the evaluation.

GDB-1 and SM-1: Control Test

Attaching `GDB` and `Scanmem` without `TyrSecure` being involved, resulted in the two tools working as intended on the game process. This is in alignment with what we expected from these two test-cases. Knowing that `Scanmem` and `GDB` worked we could start testing them against `TyrSecure`.

GDB-2 and SM-2: TyrSecure Before GDB and Scanmem

When TyrSecure was attached to the game process before attaching GDB or Scanmem, the test cases SM-2 and GDB-2 produced the same result. In both cases, we got an error that mappings for the PID of the game, was either invalid or that it did not have sufficient permission. Figure 7.1 shows the specific error message displayed when attempting to attach Scanmem to the game process. These two test cases demonstrate that when TyrSecure is attached to the game process, both Scanmem and GDB are unable to access the memory mappings of the process, and any further steps of its intended behavior. This result is in alignment with our expectations.

```
Enter the pid of the process to search using the "pid" command.  
Enter "help" for other commands.  
> pid 622  
error: failed to open maps file /proc/622/maps.  
error: sorry, there was a problem getting a list of regions to search.  
warn: the pid may be invalid, or you don't have permission.  
Enter the pid of the process to search using the "pid" command.  
Enter "help" for other commands.  
> █
```

Figure 7.1: Scanmem try to attach to game process, after eBPF is attached to the game process.

SM-3 and GDB-3: TyrSecure After GDB and Scanmem

Test case SM-3 showed that when attaching TyrSecure after Scanmem, TyrSecure started blocking Scanmem's access to memory, making Scanmem unable to read or manipulate the game memory. In Figure 7.2, we can see that Scanmem tries to open all memory regions associated with the games PID but is unsuccessful, leading to zero matches for the search. A similar result was observed for test case GDB-3, where GDB gave the error Operation not permitted when trying to access memory mappings. This shows that our eBPF program is able to block memory access even if the debugging tool is attached before TyrSecure, and the result is in alignment with our expectations.

```
75/77 searching 0x7fe060fc0000 - 0x7fe060fc1000error: unable to open /proc/622/mem.  
.ok  
76/77 searching 0x7fe061005000 - 0x7fe061006000error: unable to open /proc/622/mem.  
.ok  
77/77 searching 0x7fff636b5000 - 0x7fff636d6000error: unable to open /proc/622/mem.  
.ok  
info: we currently have 0 matches.  
0> █
```

Figure 7.2: Result of trying to isolate variable address using Scanmem after eBPF is attached.

CP-1: TyrSecure Against ac_rhack

In test case CP-1 it could be observed that the game is launched with ac_rhack, successfully. After TyrSecure was attached to the game process, ac_rhack continues working successfully, and no events were reported by TyrSecure. This indicates that at the current version of TyrSecure is unable to detect a cheating method like the one ac_rhack uses

Test Result Discussion and Conclusion

Based on the test results, we can conclude that TyrSecure, in its current state, is capable of detecting and blocking debugging utilities such as Scanmem and GDB when they attempt to perform the intended behavior defined above, but unable to detect cheating programs like ac_rhack. In none of the test cases did we experience unexpected behavior. This may reflect on the test setup used, where the first six tests using GDB and Scanmem was very simple and essentially unit test of the memory access filter of TyrSecure and ac_rhack being out of the scope of TyrSecure. Reflecting on the evaluation we might have been to optimistic with what was program would be able to do, and we could have looked into other complex cheating method that have at least some degree of relevance for malicious memory access.

Chapter 8

Discussion

In this chapter we will go over various points of discussion for the project's contents. This includes the limitations of our test, discussion about the requirements from the requirements list we did not cover, the limitations of eBPF, and a discussion of the project as a whole.

8.1 Limitations of the Test

In our test, as mentioned in chapter 7, we only tested it against two debugger tools and one cheat program. Having only one cheat program being tested, does limit the ability to conclude on the capabilities of the current state of TyrSecure especially when the method used by the cheat is not considered in the current implementation. That could beg the question why this specific cheat was chosen and why only one cheating program was chosen. This does however show one method for applying a cheating program to an actual game, which can be used to develop further on the requirements and prioritization of them for TyrSecure. When that is said we can from the test see that TyrSecure is able to monitor when debugging tools try to hook to the game process and block these actions.

The test environment as of now only focus on single player mode, even though that we in the problem analysis emphasized that detection of cheating is most relevant in a multiplayer-context, where stakes are ultimately higher. There are multiple reasons for this, with the main one being that ethical open-source cheat programs is often adapted to single player mode where the cheat essentially does no harm to other users. This could raise the concern that there are certain cheats focused on multiplayer aspects that would be harder and more time consuming to implement for testing. Therefore at this early stage of implementation this aspect was not prioritized. But for future testing hosting private servers could be used to ethically test multiplayer cheating.

8.2 Unsolved Requirements

The current Version of TyrSecure has only implemented one of the set requirements for an effective replacement of kernel modules, that being monitoring and blocking memory

access. This means that the current state of TyrSecure lacks sufficient capabilities to make a practical evaluation of its effectiveness. The reasons for only implementing that one requirement is based on the time constraint of the project and were chosen since it would be a relative simple and durable implementation. The current version of TyrSecure lays the foundation for expanding on its capabilities and implementing more of the requirements made in chapter 5.

Implementing more requirements would enable us to make a more sufficient evaluation of TyrSecure's capabilities. As of now the requirements are founded on theoretical knowledge and assumptions. Implementing them in practice and expanding the testing environment would not only enable us to confirm or dis-confirm our assumptions but could expand on requirements that has not yet been discovered. An example of this could be our testing of the open-source cheat program that displayed challenges of cheating not yet touched upon in the requirements. This leads us to the list of requirements at this point also being a bit high-level and not covering all the sufficient requirements for an actual effective replacement of kernel modules.

8.3 Limitations of eBPF

Troughout our work with eBPF, we found that while eBPF does present us with some ways to rethink anti-cheat, eBPF in its essence also comes with some possible drawbacks. One of its possible drawbacks is its incapability to ensure trust in the kernel of the device it is installed on. As seen in other anti-cheating programs, ensuring trust in the kernel is crucial for the reliability of the anti-cheat program. These programs use kernel to check for an intact chain-of-trust from boot of the device. Since eBPF programs run as a "guest" inside the kernel space, it is not possible to check. This is an area that needs more research to estimate the true capabilities of ensuring trust in the kernel.

Another thing of note is that it is unsure how well eBPF can cover some of the unexplored detection methods. Techniques such as DMA (Direct Memory Access) hardware and Hypervisor-level introspection operate outside the reach of the kernels monitoring hooks. Furthermore, because Linux is an open-source platform, advanced users can modify the kernel itself to bypass or blind eBPF probes.

As we mentioned in section 2.7.2, eBPF introduced CO-RE to enable eBPF programs to run on multiple different kernel version without modification. While this is a good thing, and also the reason why we utilize CO-RE, it comes with a drawback. Because CO-RE relies on BTF (BPF Type Format) metadata to adjust to different kernel versions, it effectively embeds a high-fidelity "roadmap" of the program's data structures directly into the binary. This metadata preserves the original names of structs and fields that would typically be lost in a standard "stripped" binary, meaning it aids people looking to reverse engineer TyrSecure. One might think that we simply then just introduce obfuscation techniques and encryption into our code to combat this. However, since the eBPF verifier must be able to prove that your program is safe, will not crash the kernel, and will always terminate, it directly clashes with how obfuscation and encryption works.

Chapter 9

Conclusion

In the project, we have explored the field of anti-cheat for video games. Here we found that a major part of the largest existing anti-cheats operate within the kernel-level, and while this makes the anti cheats cover more surfaces, kernel-level anti-cheats also lead to potential concerns for the users. Furthermore, the development of kernel modules is complex and have lead to critical failures in the past. While current kernel-level solutions are mainly made for the Windows OS, we found that outside contributors like the Valve Corporation are pushing the Linux gaming field, by enabling more gaming capabilities with the Linux OS, leading to a rise in Linux gamers. Therefore we wanted to rethink the way of doing anti-cheat, while adhering to the growing amount of Linux gamers, leading to a focus on the eBPF-functionalites that the Linux OS presents. This lead us to develop our own anti-cheat solution called TyrSecure that utilizes eBPF-functionalities to partially replace parts of kernel level anti-cheat solutions. While we sought out to answer whether eBPF could be used as an effective replacement for kernel modules, we must acknowledge that currently TyrSecure is not in a state where answering this question is possible at the moment. This conclusion boils down to TyrSecure only adhering to one of the six current requirements we deemed necessary for an eBPF-solution to effectively replace kernel modules. Therefore, more work is needed to fully conclude whether eBPF is an effective replacement.

Chapter 10

Future Work

The following sections will expand on important directions for possible future work to be done. First, we propose an architectural redesign that restructures current monitoring into two monitoring components to reduce overhead. We discuss how TyrSecure can be expanded further for internal detection capabilities. We highlight the need to harden TyrSecure against tampering and further issues that must be addressed for production ready deployment that include virtual machine detection, integration with Secure Boot and also Wine/Proton translation-layer obstacles. Together, these future works outline issues that need to be addressed to move towards a more practical Linux anti-cheat solution.

10.1 Architecture Redesign and Feature Expansion

The current design serves as a proof-of-concept for an eBPF-based anti-cheat. However, as we developed these features, we identified challenges regarding scalability and thread management. Currently, TyrSecure utilizes a dedicated user-space thread for each eBPF program's dataflow. As more features are added, this 1:1 ratio will lead to excessive resource consumption. To address this, the system must be restructured to optimize performance and align with new detection requirements.

Consolidation Into a Dual-Monitor Design

To reduce thread overhead and improve efficiency, we plan to consolidate our code into two primary programs:

- **The Always-Running Monitor:** This program will run in the background at all times, even when the game is closed. Its primary purpose is to detect "pre-loading" threats, such as malicious kernel modules or unauthorized library injections that occur before the game is launched.
- **The Real-Time Game Monitor:** This program will activate only while the game is running. It will handle all high-frequency checks such as the syscall monitoring implemented in our current version to ensure the game remains secure during active play.

By moving to these two major programs, we can significantly reduce thread usage.

Shifting From External to Internal Detection

Our current implementation successfully blocks external threats, such as third-party processes attempting to read game memory via ptrace or /proc/. However, to reach a production-ready state, the system must expand into internal detection to catch threats originating from within the game's own process space.

Future requirements for the real-time engine include:

- **Memory Protections:** Monitoring both mprotect and mmap syscalls. This allows the system to detect when a process modifies its own memory to "Executable" space or allocates new "anonymous" executable regions key indicators of manual shared objects mapping and shellcode execution.
- **Environment Integrity:** Detecting LD_PRELOAD exploits and unauthorized shared memory attachments (e.g., shmat) that allow external manipulation without direct syscall triggers.

Binary Obfuscation

Since eBPF programs are managed and loaded by user-space applications, the entire execution chain requires protection against tampering. To harden this architecture, binary obfuscation can be employed to increase the complexity of reverse engineering. By masking the logic of the loader and the eBPF map interactions, developers can make it significantly more difficult for attackers to identify, intercept, or disable the anti-cheat defense mechanisms. However, as we discussed in section 8.3 utilizing obfuscation techniques might cause conflicts with the eBPF verifier.

10.2 Virtual Machine Detection

One of the proposed requirements that was not explored throughout this project is virtual machine detection. While this was not explored in the current project, eBPF does provide us with some features that could possibly allow us to detect a virtual machine being run. One way we can do this is by attaching eBPF's kprobes to functions like `vfs_read` and `openat()`. We can then monitor for attempts to read `/sys/class/dmi/id/product_name` which, in the case that this was executed on a virtual machine, would return an emulated hardware name rather than that of actual physical hardware. Another approach is virtual machines' use of "hypercalls" to communicate with the host (for example, to sync the system clock). We can use eBPF to trace things like the `kvm_hypercall` symbol in the potential guest kernel. In the case of a kernel-based virtual machine, this function would be executed often while it would never be executed outside the guest kernel. Although these two examples are ways to utilize eBPF for virtual machine detection, it needs to be stated that there exist ways to counteract or hide from the previously mentioned methods. For the first method, a direct hardware passthrough would make it so the hardware name id would show up as that of the original machine. A way to combat the interception of

`kvm_hypercall` would be to do a small rewrite to the disk-kernel of the virtual machine, to make the hypervisor do an additional call with a new name, that does the same thing as `kvm_hypercall`.

10.3 Integration With Secure Boot

Secure Boot state is not directly accessible through TyrSecure alone, as eBPF does not provide an interface for reading its state. The next step is to develop a user-space client that acts as the core component of our anti-cheat system. The client would load/attach eBPF programs and handle tasks outside eBPF, such as integrity checks for verifying whether Secure Boot is enabled and kernel is running in Lockdown mode.

To check Secure Boot state from user space, the client can read UEFI exposed variables under `/sys/firmware/efi/efivars/`. Variables such `SecureBoot` and `SetupMode` provide information about the state. If `SecureBoot=1` and `SetupMode=0`, it means that Secure Boot is enabled and the firmware is not in Secure Boot setup mode. We can also check whether Lockdown is enabled by reading from `/sys/kernel/security/lockdown`.

By integrating these additional features to the client, we can verify that the system is running in a trusted state. This can provide a better separation of responsibilities, where eBPF is focusing on kernel-level monitoring and user-space client can verify integrity of the system.

10.4 Integration With Wine/Proton

As discussed in subsection 2.4.2, Wine and Proton have become essential to the Linux gaming ecosystem. They are used as a compatibility layer to run Windows software on Linux systems. However, they introduce significant hurdles for the eBPF-based approach used in this research as Proton translates Windows API calls into Linux system calls in real-time, producing many system calls that overlap with common cheating techniques. As a result, trying to patch around system calls that Proton makes, would mean to constantly add special cases that over time could make the anti-cheat difficult to maintain and such a solution would unlikely be capable of reliably separating real cheating and Proton behavior.

For that reason, Wine and Proton support should be treated as a new research direction rather than a feature that could be added to the current anti-cheat system. To support a translation-layer would mean that the anti-cheat should be capable of distinguishing real translation behaviour from real cheating attempts even when the syscall surface looks similar.

```
→ strace -c steam steam://rungameid/2215390 © 12:57:05 AM
```

% time	seconds	usecs/call	calls	errors	syscall
56,90	0,002330	166	14	7	wait4
11,53	0,000472	67	7		clone
7,35	0,000301	2	105		rt_sigprocmask
3,47	0,000142	12	11	8	execve
2,76	0,000113	3	32		read
2,69	0,000110	2	47		mmap
2,37	0,000097	1	51		rt_sigaction
2,27	0,000093	2	39	18	openat
1,90	0,000078	2	33		close
1,88	0,000077	3	25	8	newfstatat
0,93	0,000038	6	6		pipe2
0,76	0,000031	2	13		mprotect
0,66	0,000027	2	12		lseek
0,63	0,000026	3	7		rt_sigreturn
0,56	0,000023	1	19		fstat
0,37	0,000015	1	10		brk
0,24	0,000010	2	5	3	access
0,22	0,000009	3	3		munmap
0,22	0,000009	1	7		prlimit64
0,20	0,000008	1	5	2	ioctl
0,20	0,000008	1	6		getpid
0,20	0,000008	8	1		faccessat2
0,17	0,000007	1	6	2	fcntl
0,17	0,000007	1	6		getppid
0,12	0,000005	0	6		pread64
0,12	0,000005	2	2		getuid
0,12	0,000005	2	2		getgid
0,12	0,000005	2	2		geteuid
0,12	0,000005	2	2		getegid
0,07	0,000003	1	2		dup2
0,07	0,000003	1	2		uname
0,07	0,000003	1	2		getresuid
0,07	0,000003	1	2		getresgid
0,07	0,000003	1	3		arch_prctl
0,07	0,000003	1	2		futex
0,07	0,000003	1	3		set_tid_address
0,07	0,000003	1	3		set_robust_list
0,07	0,000003	1	3		getrandom
0,05	0,000002	1	2		getpgrp
0,05	0,000002	0	3		rseq
100,00	0,004095	8	511	48	total

Figure 10.1: Strace of proton game

As seen in Figure 10.1, analysis through tools like `strace` confirms that Proton relies on `mmap`, `mprotect`, and `execve` to function. For example, Proton uses `mprotect` to dynamically manage memory permissions for Windows binaries, an action also used by cheats to inject and override game code. Similarly, `execve` calls, which are necessary for loading the Windows environment, can resemble process spawning behavior of cheating tools.

The use of `mmap` is particularly challenging. While a cheat might use it to map a malicious library into the game’s memory space, Proton uses it to map legitimate game files and create the virtual memory layout required by Windows. This overlap makes current eBPF tracking methods inaccurate, as the anti-cheat must account for false positives generated by the translation layer itself.

Future development should consider Wine and Proton support as a new research direction for exploring the following strategies to reduce potential false positives caused by the translation layer:

- **Native Enforcement:** The simplest approach would be to assume that the game developer provides a native Linux build. In this scenario, the anti-cheat would be updated to detect if the game is being launched through Proton or Wine and, if detected, block execution. This ensures that the eBPF hooks only monitor predictable, native system calls.
- **Integrated Translation Monitoring:** A more sophisticated solution would involve building detection software inside the Wine/Proton environment itself. This internal software would be designed to identify suspicious activity from within the translation layer, similar to some of the requirements in chapter 5.

- **Custom Proton Builds:** Another solution would be to develop a custom version of Proton specifically designed to work with the anti-cheat. This "hardened" version of Proton could include built-in telemetry that "tags" every system call it makes. When the eBPF hook detects a suspicious mprotect or mmap call, it could check for the tags provided by the custom Proton build. This would allow an anti-cheat to whitelist legitimate translation traffic while remaining sensitive to any untagged calls from cheats.

10.5 Priorities for Future Development

In the above sections, we have provided a list of possible areas of future work required to achieve a practical solution. Not all of them can be implemented simultaneously, so they should be prioritized.

The highest priority is to redesign our application to have a scalable core for TyrSecure by restructuring monitoring into a dual-monitor setup for always-running monitor and a real-time game monitor. A user-space client should be a central component for loading and attaching programs, since it can do many integrity checks that cannot be handled inside eBPF alone.

Once architecture is redesigned, the next priority is to expand internal detection coverage for catching threats within the game's own process space. Monitoring modifications of memory and environment integrity checks such as LD_PRELOAD should be implemented to prevent common attack techniques.

After expanding on detection coverage, its important to ensure that system is operating in a trusted state to increase the difficulty of kernel level cheating. Secure Boot and Kernel Lockdown verification should be integrated in the user-space client. As part of operating in a trusted environment, detection of a VM should also be considered in this stage to detect whether the system is running in a virtualized environment. In addition, binary obfuscation can be applied to harden and reduce the surface of tampering with the anti-cheat.

Wine/Proton integration should be approached as a separate research direction, after core architecture and baseline detections are stable. Such research would be valuable in adding support to more variety of games that are run using the translation layer.

Altogether, prioritization of features provides a development path for further progression of Linux anti-cheat solution.

Bibliography

- [1] Alcsaar. "Riot vanguard is absurdly invasive and doesn't even accomplish its goal and never will." [Online]. Available: https://www.reddit.com/r/riotgames/comments/1g7duse/riot_vanguard_is_absurdly_invasive_and_doesnt/.
- [2] AlSch092, *Ultimateanticheat*, <https://github.com/AlSch092/UltimateAntiCheat>.
- [3] R. Alsowaigh, "Crowdstrike causes global microsoft outage: A case study," *Journal of Information Security and Cybercrimes Research*, vol. 8, pp. 01–16, Jun. 2025. doi: 10.26735/QHDD4798. [Online]. Available: https://www.researchgate.net/publication/393170974_CrowdStrike_Causes_Global_Microsoft_Outage_A_Case_Study.
- [4] Android Open Source Project, *Generic kernel image (gki) project and release builds*, <https://source.android.com/docs/core/architecture/kernel/generic-kernel-image>, Last updated: 20251009; accessed 20251029, 2025.
- [5] Android Open Source Project, *Verified boot android*, <https://source.android.com/docs/security/features/verifiedboot>, Last updated: 20240826; accessed 20251029, Aug. 2024.
- [6] Arch Linux Wiki contributors, *Kernel module*, https://wiki.archlinux.org/title/Kernel_module, Arch Wiki, accessed on 2026-01-07, Arch Linux, 2026.
- [7] Arch Linux Wiki contributors, *Signed kernel modules*, https://wiki.archlinux.org/title/Signed_kernel_modules, Arch Wiki, accessed on 2026-01-07, Arch Linux, 2026.
- [8] S. Berger. "Today i learned kernel.modules_aisabled." dfir.ch blog post, accessed on 2026-01-07. [Online]. Available: https://dfir.ch/posts/today_i_learned_lkm_kernel_modules_disabled/.
- [9] A. Boldi and A. Rapp, "Is It Legit, To You? An Exploration of Players Perceptions of Cheating in a Multiplayer Video Game: Making Sense of Uncertainty," *International Journal of HumanComputer Interaction*, vol. 40, pp. 1–21, Apr. 2023. doi: 10.1080/10447318.2023.2204276. [Online]. Available: <https://doi.org/10.1080/10447318.2023.2204276>.
- [10] M. Brengel, M. Backes, and C. Rossow, "Detecting Hardware-Assisted Virtualization," in *Proceedings of the Conference on Detection of Intrusions and Malware & Vulnerability Assessment (DIMVA)*, PDF freely available at <https://www.christian-rossow.de/publications/detectvt-dimva2016.pdf>, publications: <https://cispa.saarland/group/rossow/research>, 2016.
- [11] M. Caruso. "Cheating is on the rise in multiplayer games, and it could spell disaster for competitive esports." [Online]. Available: <https://esportsinsider.com/cheating-in-multiplayer-games-and-esports-impact>.
- [12] Cheat Engine. "Cheat engine - homepage." [Online]. Available: <https://www.cheatengine.org/>.

- [13] C. R. China and M. Goodwin. "What is eBPF?" Accessed: Dec. 10, 2025. [Online]. Available: <https://www.ibm.com/think/topics/eBPF>.
- [14] CrowdStrike. "CrowdStrike - homepage." [Online]. Available: <https://www.crowdstrike.com/en-us/>.
- [15] Cyient, "Secure boot: Foundation for the security architecture of a device," Jan. 2024, Accessed: 2025-11-24. [Online]. Available: <https://www.cyient.com/whitepaper/on-secure-boot-foundation-for-the-security-architecture-of-a-device>.
- [16] Cymulate. "Ransomware actor abuses genshin impact anti-cheat driver to kill antivirus." [Online]. Available: <https://cymulate.com/threats/ransomware-actor-abuses-genshin-impact-anti-cheat-driver-to-kill-antivirus/>.
- [17] DMA Cheats. "Dma cheats - homepage." [Online]. Available: <https://www.dma-cheats.com/>.
- [18] C. Dorner and L. D. Klausner, *If it looks like a rootkit and behaves like a rootkit: A critical examination of kernel-level anti-cheat systems*, arXiv preprint, 2024. doi: 10.48550/arXiv.2408.00500. eprint: 2408.00500. [Online]. Available: <https://arxiv.org/abs/2408.00500>.
- [19] H. B. Duh and V. H. H. Chen, "Cheating behaviors in online gaming," in *Online Communities and Social Computing (OCSC 2009)*, 2009, pp. 567–573. doi: 10.1007/978-3-642-02774-1_61.
- [20] eBPF.io. "What is eBPF? an introduction and deep dive into the eBPF technology," Accessed: Dec. 10, 2025. [Online]. Available: <https://ebpf.io/what-is-ebpf/>.
- [21] ekianjo. "Windows games compatibility on linux is at an all-time high." First published 2025-10-26; last modified 2025-10-26 13:36:01 UTC, Boiling Steam, Accessed: Oct. 30, 2025. [Online]. Available: <https://boilingsteam.com/windows-games-compatibility-on-linux-is-at-a-all-time-high/>.
- [22] Epic Games, *Eos - anti-cheat interfaces*, <https://dev.epicgames.com/docs/game-services/anti-cheat>, Accessed: 2025-10-30, 2025.
- [23] Esports Charts, *Top esports games of 2024 by prize pool / popularity*, <https://escharts.com/top-games?year=2024>, Accessed: 2025-10-29, 2024.
- [24] N. Esposito, "A short and simple definition of what a videogame is," Jan. 2005. doi: 10.26503/dl.v2005i1.177.
- [25] T. Garfinkel, K. Adams, A. Warfield, and J. Franklin, "Compatibility is not transparency: VMM detection myths and realities," in *11th Workshop on Hot Topics in Operating Systems (HotOS XI)*, San Diego, CA: USENIX Association, 2007. [Online]. Available: https://www.usenix.org/legacy/event/hotos07/tech/full_papers/garfinkel/garfinkel_html/paper.html.
- [26] J. Hamari and M. Sjöblom, "What is esports and why do people watch it?" *Internet Research*, vol. 27, no. 2, pp. 211–232, 2017. doi: 10.1108/IntR-04-2016-0085.
- [27] S. S.-A. Hannes Kunstner Mathias Højte Olsen, *From exploit to enforcement*, May 2024. Accessed: Nov. 24, 2025. [Online]. Available: https://vbn.aau.dk/ws/files/718689562/From_Exploit_to_Enforcement__Dissecting_tcyient_secure_boot_foundation_2024he_Evolution_of_Video_Game_Cheating_and_Anti_Cheat_Technologies.pdf.
- [28] IamSanjid, *Linux-process_mem: Simple library to read and write memory of a linux process through custom kernel module*, https://github.com/IamSanjid/linux-process_mem, GitHub repository, accessed on 2026-01-07, GitHub, 2020.

- [29] iBotPeaches (Apktool Project), *Apktool - a tool for reverse engineering android apk files*, <https://apktool.org/>, Accessed: 2025-11-05, 2025.
- [30] Irdeto, *Is your mobile game immune from cheating?* <https://irdeto.com/blog/is-your-mobile-game-immune-from-cheating>, Accessed: 2025-10-29, Oct. 2022.
- [31] T. Joelsson, S. Hyrynsalmi, and S. Molenaar, "The role of prosumers in the evolution of a software ecosystem: Case steam," Dec. 2018.
- [32] P. Karkallis and J. Blasco, *Vic: Evasive video game cheating via virtual machine introspection*, 2025. arXiv: 2502.12322 [cs.CR]. [Online]. Available: <https://arxiv.org/abs/2502.12322>.
- [33] B. Kordy, S. Mauw, S. Radomirovi, and P. Schweitzer, "Attack-defense trees," *Journal of Logic and Computation*, vol. 24, no. 1, pp. 55–87, Jun. 2012, ISSN: 0955-792X. DOI: 10.1093/logcom/exs029. eprint: <https://academic.oup.com/logcom/article-pdf/24/1/55/3749013/exs029.pdf>. [Online]. Available: <https://doi.org/10.1093/logcom/exs029>.
- [34] P. Koskinas. "X post without a title." [Online]. Available: <https://x.com/detecccphilippe/status/1954976139530866904>.
- [35] G. Kroah-Hartman. "The linux kernel: Stable api nonsense," The Linux Kernel Documentation, Accessed: Oct. 31, 2025. [Online]. Available: <https://www.kernel.org/doc/Documentation/process/stable-api-nonsense.rst>.
- [36] N. Kshetri, "Diffusion pattern of linux: An assessment on major technology dimensions," *First Monday*, vol. 10, Apr. 2005. DOI: 10.5210/fm.v10i8.1263.
- [37] E. Kuiter, C. Sundermann, T. Thüm, T. Hess, S. Krieter, and G. Saake, "How configurable is the linux kernel? analyzing two decades of feature-model history," *ACM Trans. Softw. Eng. Methodol.*, Apr. 2025, Just Accepted, ISSN: 1049-331X. DOI: 10.1145/3729423. [Online]. Available: <https://doi.org/10.1145/3729423>.
- [38] S. Lakshmi Narasimhan, "Remote attestation of user-space applications at runtime," Master's Programme in Security and Cloud Computing, Master's thesis, Aalto University, School of Science, Espoo, Finland, Sep. 2025, p. 58. [Online]. Available: <https://urn.fi/URN:NBN:fi:aalto-202510208274>.
- [39] Laserface. "Riot vanguard faq (league of legends)." [Online]. Available: <https://support-leagueoflegends.riotgames.com/hc/en-us/articles/24169857932435-Riot-Vanguard-FAQ-League-of-Legends>.
- [40] J. Lee et al., "Bootkitty: A stealthy bootkit-rootkit against modern operating systems," in *Proceedings of the 19th USENIX Conference on Offensive Technologies*, ser. WOOT '25, Seattle, WA, USA: USENIX Association, 2025, ISBN: 978-1-939133-50-2.
- [41] R. A. Lee, *Mobile games statistics 2025: Demographics, monetization, and market leaders*, <https://sqmagazine.co.uk/mobile-games-statistics/>, Accessed: 2025-10-27, Oct. 2025.
- [42] D. Liu, X. Gao, M. Zhang, H. Wang, and A. Stavrou, "Detecting passive cheats in online games via performance-skillfulness inconsistency," in *2017 47th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2017, pp. 615–626. DOI: 10.1109/DSN.2017.20.
- [43] LordShen. "Invisible power: Loadable kernel module (lkm)." Medium article, accessed on 2026-01-07. [Online]. Available: <https://medium.com/@lordshen/invisible-power-loadable-kernel-module-lkm-30c7aa52d19d>.

- [44] Magisk Manager Website, *Magisk manager - download magisk latest version for android*, <https://magiskmanager.com/>, Accessed: 2025-10-29, 2025.
- [45] man-pages Project, *Ptrace(2) linux manual page*, Online; accessed 2026-01-07, Linux Programmers Manual, 2025. [Online]. Available: <https://man7.org/linux/man-pages/man2/ptrace.2.html>.
- [46] Market.us, *Global mobile gaming market size, share analysis report mobile gaming market*, <https://market.us/report/mobile-gaming-market/>, Report ID: 149942, Jun. 2025.
- [47] MediaCat. "Dentsu: gaming is bigger than music and movies combined." [Online]. Available: <https://mediacat.uk/dentsu-gaming-is-bigger-than-music-and-movies-combined/>.
- [48] Microsoft. "Trusted platform module technology overview." [Online]. Available: <https://learn.microsoft.com/en-us/windows/security/hardware-security/tpm/trusted-platform-module-overview>.
- [49] Microsoft. "What's a trusted platform module (tpm)?" Accessed: Oct. 31, 2025. [Online]. Available: <https://support.microsoft.com/en-us/topic/what-s-a-trusted-platform-module-tpm-705f241d-025d-4470-80c5-4feeb24fa1ee>.
- [50] National Vulnerability Database. "Cve-2020-36603 detail." [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2020-36603>.
- [51] Ngày cp nht gn nht, *7 reasons causing users to abandon windows*, <https://mytour.vn/en/blog/bai-viet/7-reasons-causing-users-to-abandon-windows.html>, Accessed: 2025-10-31, 2025.
- [52] K. Orland. "Blizzard fixes diablo iii gold duplication bug, but the damage may be done." [Online]. Available: <https://arstechnica.com/gaming/2013/05/blizzard-fixes-diablo-iii-gold-duplication-bug-but-the-damage-may-be-done/>.
- [53] M. Park. "Battlefield 6 and valorant's invasive anti-cheats are locked in a turf war." [Online]. Available: <https://www.pcgamer.com/games/fps/battlefield-6-and-valorants-invasive-anti-cheats-are-locked-in-a-turf-war/>.
- [54] A. Philbert, *Kernel-level anti-cheat: The necessary evil or a misstep in gaming security?* <https://medium.com/@Pheelbert/kernel-level-anti-cheat-the-necessary-evil-or-a-misstep-in-gaming-security-365bfcf3fddf>, Accessed: 2025-10-27, 2024.
- [55] S. Rahman, *Pinning user-space pages in the linux kernel*, Accessed: 2025-12-28. [Online]. Available: <https://blogs.oracle.com/linux/pinning-userspace-pages-in-the-linux-kernel>.
- [56] Redfox Security. "Exploiting linux capabilities: CAP_SYS_MODULE." Medium article, accessed on 2026-01-07. [Online]. Available: <https://redfoxsecurity.medium.com/exploiting-linux-capabilities-cap-sys-module-055cac48caab>.
- [57] rhboot contributors, *Shim, a first-stage uefi bootloader*, <https://github.com/rhboot/shim>, GitHub repository. Latest release: 16.1 (Aug 13, 2025). Accessed: 2025-12-17, 2025.
- [58] Riot Games, */dev: Vanguard x lol retrospective*, <https://www.leagueoflegends.com/en-us/news/dev/dev-vanguard-x-lol-retrospective/>, Accessed: 2025-10-30, 2024.
- [59] Riot Games. "A message about vanguard from our security & privacy teams," Accessed: Oct. 31, 2025. [Online]. Available: <https://www.riotgames.com/en/news/a%2Dmessage%2Dabout%2Dvanguard%2Dfrom%2Dour%2Dsecurity%2Dprivacy%2Dteams>.

- [60] Riot Games. "Riot games - homepage." [Online]. Available: <https://www.riotgames.com/en>.
- [61] Rootkit Hunter Development Team, *Rootkit hunter*, <https://rkhunter.sourceforge.net/>, Official project site, accessed on 2026-01-07, SourceForge, 2026.
- [62] H. K. Ryan Soliven. "Ransomware actor abuses genshin impact anti-cheat driver to kill antivirus." [Online]. Available: https://www.trendmicro.com/en_us/research/22/h/ransomware-actor-abuses-genshin-impact-anti-cheat-driver-to-kill-antivirus.html.
- [63] Scannells, *Ac_rhack*. Accessed: Nov. 24, 2025. [Online]. Available: https://github.com/scannells/ac_rhack.
- [64] A. Schlögl, S. Elefante, and V. Hodirna, "Running windows-applications on a linux hpc cluster using WINE," in *ASHPC23 – Austrian-Slovenian HPC Meeting 2023*, Conference dates: 2023-06-13–2023-06-15; IST-REx-ID: 13161; Acknowledgement: Thanks to Jesse Hansen for suggestions on improving the abstract., Maribor, Slovenia: EuroCC, Jul. 2023, p. 59.
- [65] T. Schöber and G. Stadtmann, "The dark side of e-sports: An analysis of cheating, doping & match-fixing activities and their countermeasures," *International Journal of Esports*, vol. 1, no. 16, 2022, CC BY. [Online]. Available: <https://www.ijesports.org/article/98/html>.
- [66] E. Steinbrink and C. Reuter, "The impact of transparency and trust on user acceptance of contact tracing apps: Implications for the adoption of crisis response apps," *International Journal of Disaster Risk Reduction*, vol. 112, p. 104661, 2024, issn: 2212-4209. doi: <https://doi.org/10.1016/j.ijdr.2024.104661>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2212420924004230>.
- [67] I. S. Stephanie Susnjara. "What is a hypervisor?" Accessed: Dec. 21, 2025. [Online]. Available: <https://www.ibm.com/think/topics/hypervisors>.
- [68] J. Stühn, J.-N. Hilgert, and M. Lambertz, "The hidden threat: Analysis of linux rootkit techniques and limitations of current detection tools," *Digital Threats*, vol. 5, no. 3, Oct. 2024. doi: 10.1145/3688808. [Online]. Available: <https://doi.org/10.1145/3688808>.
- [69] N. TAKAHASHI and T. TAKAMATSU, "Unix license makes linux the last major piece of the puzzle," *Annals of Business Administrative Science*, vol. 12, no. 3, pp. 123–137, 2013. doi: 10.7880/abas.12.123.
- [70] B. Taneja, *Structure of Linux operating system*, Accessed: 2025-12-23. [Online]. Available: <https://drbtaneja.com/structure-of-linux-operating-system>.
- [71] S. Tang. "Most popular esports in 2025: 5 most played games right now." [Online]. Available: <https://esportsinsider.com/most-popular-esports-games-2025>.
- [72] Tencent Cloud, *Ace - game security solution*, <https://www.tencentcloud.com/products/ace>, Accessed: 2025-10-29, 2025.
- [73] L. B. The and V. N. Khanh, "Gameguard: A windows-based software architecture for protecting online games against hackers," in *Proceedings of the 1st Symposium on Information and Communication Technology*, ser. SoICT '10, Hanoi, Vietnam: Association for Computing Machinery, 2010, pp. 171–178, ISBN: 9781450301053. doi: 10.1145/1852611.1852643. [Online]. Available: <https://doi-org.zorac.aub.aau.dk/10.1145/1852611.1852643>.

- [74] The Linux Foundation. "Linux foundation research shows economic value of open source software rising in terms of benefits vs. costs," Accessed: Oct. 31, 2025. [Online]. Available: https://www.linuxfoundation.org/press/linux-foundation-research-shows-economic-value-of-open-source-software-rising-in-terms-of-benefits-vs-costs?utm_source=chatgpt.com.
- [75] S. Tulach. "The issue of anti-cheat on linux," Accessed: Oct. 31, 2025. [Online]. Available: <https://tulach.cc/the-issue-of-anti-cheat-on-linux/>.
- [76] Ubuntu. "Uefi secure boot," Accessed: Nov. 24, 2025. [Online]. Available: https://documentation.ubuntu.com/security/security-features/platform-protections/secure-boot/?utm_source=chatgpt.com.
- [77] Uplatz Blog. "Ebpf: A comprehensive analysis of the inkernel programmability revolution," Accessed: Dec. 10, 2025. [Online]. Available: <https://uplatz.com/blog/ebpf-a-comprehensive-analysis-of-the-in-kernel-programmability-revolution/>.
- [78] Valve Corporation, *Steam hardware & software survey: Welcome to steam*, <https://store.steampowered.com/hwsurvey/Steam-Hardware-Software-Survey-Welcome-to-Steam>, Accessed: 2025-10-27, 2025.
- [79] B. van de Ven, "Cheating and anti-cheat system action impacts on user experience," 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:259362902>.
- [80] R. Wang and Y. Yan, "A survey of secure boot schemes for embedded devices," in *2022 24th International Conference on Advanced Communication Technology (ICACT)*, 2022, pp. 224–227. doi: 10.23919/ICACT53585.2022.9728840.
- [81] Wikipedia contributors. "Proton (software)." Page last edited on 28 October 2025 (UTC), Wikipedia, Accessed: Oct. 30, 2025. [Online]. Available: [https://en.wikipedia.org/wiki/Proton_\(software\)](https://en.wikipedia.org/wiki/Proton_(software)).
- [82] R. Wilkins. "The chain of trust. "[Online]. Available: https://uefi.org/sites/default/files/resources/UEFI%20Forum%20White%20Paper%20-%20Chain%20of%20Trust%20Introduction_Final.pdf.
- [83] M. Wilson. "Riot dev explains valorants anti-cheat kernel driver and why it activates when you boot your pc," Accessed: Dec. 16, 2025. [Online]. Available: <https://www.kitguru.net/gaming/matthew-wilson/riot-dev-explains-valorants-anti-cheat-kernel-driver-and-why-it-activates-when-you-boot-your-pc/>.